

Estándar IEEE 16326 – ingeniería de sistemas y software – Proceso del ciclo de vida – Gestión de proyectos

Administración de proyectos 1

Integrantes

Ramírez	García	Oswaldo
Rios	Carrera	Jesús
Vargas	Angeles	Uriel
Osornio	Tapia	Jorge
Pérez Estrada	Guadalupe	Yosabeth

Matriculas

21-011-1167
19-011-0599
19-011-1318

21-011-0194
20-011-0820

Nombre del profesor
Máximo Eduardo Sánchez Gutiérrez

Historial de cambios

Versión	Fecha	Páginas afectadas	Descripción del cambio	Autor
1.0	02-03-2026	Todo el documento	Creación inicial del PMP	Jorge Tapia
1.1	17-03-2026	Secciones 5,6 y 7	Se agregó información del Sprint 2	Jorge Tapia
1.2	20-03-2026	Sección 8	Se integro sprint retrospective y gestión de riesgos	Jorge Tapia
1.3	22-03-2026	Sección 8.7	Se agregaron métricas (KPIs)	Jorge Tapia
1.4	24-04-2026	Sección 5.2	Se Integraron actividades WBS	

Prefacio

El presente documento describe el Plan de Gestión del Proyecto (PMP) para el desarrollo de un sistema de comanda digital. Este documento está dirigido a todos los involucrados en el proyecto, incluyendo el equipo de desarrollo, stakeholders y responsables de la gestión del proyecto. Su propósito es establecer la organización, planificación, ejecución y control del proyecto, siguiendo buenas prácticas y estándares internacionales.

Contenido

1.	Vision general del proyecto	3
1.1	Resumen del proyecto	3
1.1.2	Supuestos y restricciones	3
1.1.3	Entregables del proyecto	3
1.1.4	Cronograma y presupuesto	3
1.2	Evolución del plan	3
2.	Referencias	3
3.	Definiciones	3
4.	Contexto del Proyecto	3
4.1	Modelo de proceso	3
4.2	Mejora del proceso	3
4.3	Infraestructura	3
4.4	Métodos y herramientas	3
4.5	Plan de aceptación	3
4.6	Organización del proyecto	3
5.	Planificación del proyecto	4
5.1	inicialización	4
5.1.1	Plan de estimación (aquí va esta entrega)	4
5.1.2	Plan de personal	4
5.1.3	Recursos	4
5.1.4	Entrenamiento	4
5.2	Planes de trabajo	4
5.2.1	Actividades (WBS)	4
5.2.2	Cronograma y presupuesto	4
6.	Evaluación y contro	4
6.1	Gestión de requisitos	4
6.2	Control de alcance	4
6.3	Control de cronograma	4
6.4	Control de presupuesto	4
6.5	Aseguramiento de la calidad	4

6.6 Subcontratistas	4
6.7 Cierre del proyecto	4
7. Entrega del proyecto	4
8. Procesos de soporte	4
8.1 Supervisión y entorno de trabajo	4
8.2 Gestión de decisiones	4
8.3 Gestión de riesgos	4
8.4 Configuración (Git, versiones)	4
8.5 Gestión de la información	4
8.6 Aseguramiento de calidad (QA)	4
8.7 Medición	4
8.8 Revisiones y auditorias	5
8.9 Verificación y validación	5
9. Planes adicionales	5

1. Visión general del proyecto

1.1 Resumen del proyecto

El **Sistema de Comanda Digital** surge como una solución tecnológica integral para la modernización de la industria restaurantera. El proyecto no solo busca digitalizar el papel, sino optimizar el ecosistema de comunicación entre tres pilares fundamentales: **Cliente (Comensal)**, **Producción (Cocina/Barra)** y **Administración (Caja/Gerencia)**. Mediante una arquitectura cliente-servidor, el sistema permite la toma de pedidos de forma autónoma por parte del cliente a través de dispositivos móviles o tablets situadas en las mesas. Esta información viaja en tiempo real hacia una interfaz de gestión en cocina, eliminando los "cuellos de botella" informativos, reduciendo el margen de error humano en un 90% y agilizando el tiempo de respuesta del servicio.

Problemas

La dependencia de comandas físicas genera lentitud en horas pico, errores en la preparación por letra ilegible y una desconexión entre el inventario real y las ventas registradas.

Solución

Una plataforma desarrollada en **PHP y SQL** que permite la toma de pedidos mediante dispositivos móviles, con actualización automática en pantallas de cocina y un dashboard administrativo para el control de inventarios y reportes financieros.

1.1.1 Propósito, alcance y objetivo

La norma IEEE 16326 exige definir claramente qué se va a hacer (propósito), hasta dónde llega el proyecto (alcance) y qué se espera lograr (objetivos) para evitar la expansión descontrolada del alcance (*creeping scope*). En perfecta concordancia con el estándar de descripción arquitectónica **ISO/IEC/IEEE 42010:2011**, esta delimitación formaliza la estructura base del sistema y establece un lenguaje común entre todos los *stakeholders* (desarrolladores, personal operativo del restaurante y administradores), asegurando que la solución evolucione de forma ordenada, escalable y sin desviarse de las necesidades prioritarias del negocio.

PROPÓSITO

El propósito de este proyecto es desarrollar e implementar la arquitectura del **Sistema de Comanda Digital (SCD)**, un sistema web orientado a la digitalización y optimización del proceso de toma y gestión de pedidos dentro del restaurante. El sistema busca sustituir de manera definitiva las comandas físicas, reducir

drásticamente los errores de comunicación y optimizar los tiempos de servicio mediante una guía técnica estructurada que abarque desde la interacción del cliente en la mesa hasta la coordinación en cocina y la validación en caja.

ALCANCE El alcance de este proyecto se centra exclusivamente en el diseño y desarrollo arquitectónico de la aplicación web responsiva adaptada a las operaciones internas del restaurante.

Dentro del alcance:

- **Toma de pedidos en mesa:** Interfaz web responsiva diseñada para ejecutarse en *tablets* instaladas en cada mesa, incorporando una identificación única por dispositivo (Id_Mesa).
- **Personalización de pedidos:** Módulo interactivo que permite al cliente modificar sus platillos en tiempo real (agregar o quitar ingredientes, especificar cantidades y añadir comentarios especiales).
- **Comunicación en tiempo real:** Flujo síncrono de envío de datos desde las *tablets* de las mesas hacia una pantalla de visualización centralizada en el área de cocina.
- **Módulo de caja y pre-facturación:** Generación e impresión de tickets físicos para la validación del pedido y cobro en caja previo a la preparación en cocina.
- **Función de asistencia digital:** Herramienta integrada en la interfaz de la *tablet* para que el cliente solicite el apoyo presencial de un mesero de forma inmediata.
- **Gestión administrativa básica:** Panel de control para el administrador orientado al seguimiento básico del inventario de insumos y generación de reportes operativos.
- **Aseguramiento de calidad y cierre:** Ejecución de pruebas unitarias, de integración y de aceptación por cada componente de la arquitectura, acompañadas de su respectiva documentación técnica y manuales de usuario.

Fuera del alcance (Lo que NO se incluye):

- **Procesamiento de pagos en línea:** No se contempla la integración con pasarelas de pago externas (Stripe, PayPal) ni sistemas bancarios; el registro de cobro se gestionará de manera física/manual en caja tras la impresión del ticket.
- **Análítica empresarial avanzada:** Exclusión de funciones complejas de *Business Intelligence* (BI) o integraciones con sistemas externos de gestión empresarial (como ERPs o CRMs comerciales).
- **Desarrollo móvil nativo:** El sistema no contará con aplicaciones instalables para iOS o Android; su ejecución se limitará estrictamente a navegadores web modernos para garantizar compatibilidad multiplataforma.

- **Infraestructura en la nube compleja:** El despliegue inicial está diseñado para un entorno de servidor local (Apache/MySQL), por lo que se excluyen arquitecturas auto-escalables en la nube (AWS, Azure).

OBJETIVOS Los objetivos del proyecto se definen bajo el criterio SMART (Específicos, Medibles, Alcanzables, Relevantes y Temporales):

1. **Eficiencia en el servicio:** Reducir en un 30% los tiempos de espera del cliente en comparación con el método tradicional de comandas de papel, medido desde el envío en la *tablet* hasta la entrega del platillo.
2. **Mitigación de errores operativos:** Reducir en un 90% los errores de transcripción y comunicación interna en la cocina provocados por letra ilegible o malentendidos verbales.
3. **Trazabilidad total de consumos:** Garantizar el 100% de la correlación de los pedidos con su mesa de origen mediante la correcta vinculación del identificador único de dispositivo (Id_Mesa).
4. **Estabilidad del Software:** Mantener una tasa de error (*Bug Rate*) inferior al 20% durante las pruebas de aceptación técnica al cierre de cada ciclo de desarrollo.
5. **Cumplimiento del Cronograma:** Completar y entregar el sistema web completamente funcional en un periodo cerrado de **8 sprints (16 semanas)**, concluyendo de manera definitiva en junio de 2026.
6. **Rendimiento del equipo:** Mantener una velocidad de desarrollo promedio constante de entre 25 y 30 *Story Points* por sprint.

1.1.2 Supuestos y restricciones

Restricciones

Las restricciones representan limitaciones reales, técnicas, organizacionales o legales impuestas al proyecto que el equipo de desarrollo no puede modificar. Su incumplimiento compromete directamente la viabilidad del sistema.

Categoría	Descripción de la restricción	Consecuencia de incumplimiento
Tecnológica	El sistema debe ejecutarse obligatoriamente sobre el <i>stack</i> local compuesto por PHP 8.2, MySQL 8.0 y Apache 2.4 . No se permite el uso de otros lenguajes	Incompatibilidad con el entorno de despliegue final en el restaurante y rechazo técnico de la entrega.

	de backend o motores de bases de datos.	
Temporal	El proyecto cuenta con un plazo estricto e inamovible de 8 sprints (16 semanas) , iniciando el 02 de marzo de 2026 y finalizando el 28 de mayo de 2026 .	Reprobación académica del proyecto de certificación y penalización por incumplimiento de cronograma ante los <i>stakeholders</i> .
Presupuestaria	El presupuesto financiero asignado al proyecto es de \$0.00 MXN . Se restringe el desarrollo al uso exclusivo de tecnologías de código abierto, <i>software</i> libre y hardware preexistente.	Detención del desarrollo por incapacidad de adquirir licencias comerciales o servicios de pago.
Seguridad y Privacidad	Es obligatorio implementar un control de acceso basado en roles (RBAC) estricto. Toda la información de comandas y ventas debe almacenarse localmente, prohibiendo explícitamente el uso de servicios de almacenamiento en la nube.	Vulnerabilidad ante accesos no autorizados a funciones críticas.
Personal	El equipo está limitado estrictamente a 3 desarrolladores con roles polivalentes , disponiendo de un máximo de 5 horas diarias por integrante para el proyecto.	Reducción de la velocidad del sprint (<i>Sprint Velocity</i>), sobrecarga de trabajo y omisión forzada de entregables planificados.
Operativa / Usabilidad	El diseño de la interfaz web debe ser lo suficientemente intuitivo para que el personal de cocina y meseros no requiera más de 30 minutos de capacitación	Resistencia al uso por parte del personal del restaurante, ralentización del servicio y aumento de errores humanos en la gestión de comandas.

		total para operar el sistema con fluidez.	
Cumplimiento de Estándares		El diseño arquitectónico está obligado a seguir estrictamente la estructura de vistas y puntos de vista dictados por el estándar ISO/IEC/IEEE 42010:2011 .	El documento perderá su validez de certificación y consistencia formal ante los evaluadores del proyecto.
Compatibilidad de Hardware (Clientes)		Las <i>tablets</i> de las mesas y la pantalla de cocina deben contar con navegadores modernos que soporten de forma nativa HTML5, CSS3 y JavaScript moderno (ES6+) .	Fallos en la renderización de la interfaz, pérdida de responsividad y mal funcionamiento de los scripts en tiempo real.
Dependencia del Entorno		El sistema debe ser capaz de operar al 100% de su capacidad sin requerir conexión a internet externa , dependiendo únicamente de la red de área local (LAN).	Caída total del sistema si el backend intenta consumir dependencias, tipografías o librerías mediante CDNs externas en lugar de locales.
Legal / Normativa		El sistema debe cumplir con la normativa local de protección de datos (ej. LFPDPPP en México) si se almacena información de empleados, y no debe registrar datos fiscales regulatorios complejos al estar fuera del alcance de facturación.	Sanciones legales por mal manejo de datos de usuarios o malentendidos sobre la validez fiscal de los tickets impresos.

Supuestos

Los supuestos son factores o premisas que el equipo de desarrollo da por verdaderos para fines de planificación, pero que están fuera de su control directo. Si un supuesto resulta ser falso, se transforma inmediatamente en un riesgo para el proyecto.

Supuesto de Proyecto	Impacto si resulta falso	Riesgo asociado en §8.3
----------------------	--------------------------	-------------------------

Infraestructura de Red: El restaurante dispondrá de una red Wi-Fi local dedicada, estable y con cobertura total tanto en el área de mesas como en la cocina central.	Pérdida de la sincronización en tiempo real de los pedidos; las <i>tablets</i> no podrán enviar las comandas a la pantalla de cocina ni a la caja.	R-01: Interrupción del servicio por fallo en la infraestructura de red local del cliente.
Hardware de Servidor: El cliente proveerá un equipo de cómputo que funcionará como servidor local con las especificaciones mínimas de hardware necesarias para ejecutar Apache y MySQL de forma fluida.	Degradación severa del rendimiento del sistema, latencia alta en la respuesta de las <i>tablets</i> y caídas constantes del servicio bajo alta demanda.	R-02: Incompatibilidad o insuficiencia del hardware del servidor local del restaurante.
Disponibilidad del Equipo: Todos los integrantes del equipo mantendrán una disponibilidad constante y un estado de salud óptimo durante las 16 semanas de ejecución del proyecto.	Aparición de cuellos de botella en módulos críticos, sobrecarga de tareas en los miembros restantes y retraso en el cierre de los sprints.	R-03: Ausentismo o baja imprevista de un miembro del equipo de desarrollo.
Hardware de Pruebas: Se dispondrá de al menos una <i>tablet</i> física funcional desde el inicio de las etapas de pruebas para validar la responsividad y la asignación del Id_Mesa.	Imposibilidad de validar la usabilidad real del sistema en producción, generando fallos ocultos de visualización o interacción en los dispositivos finales.	R-04: Carencia de dispositivos físicos para pruebas de aceptación en entorno real.
Entrega de Información (Menú): El administrador del restaurante proporcionará la lista completa de productos, precios, categorías e insumos a más tardar al finalizar el Sprint 2.	Retraso en la carga de la base de datos de prueba, impidiendo validar el comportamiento real del menú digital en los sprints intermedios.	R-05: Retraso en entregables por falta de insumos de información por parte del cliente.
Suministro Eléctrico Estable: El restaurante cuenta con una instalación eléctrica regulada o un sistema de	Un apagón corromperá la base de datos de MySQL en pleno servicio, provocando pérdida de	R-06: Pérdida de integridad de datos por fallos en el suministro eléctrico local.

energía ininterrumpida (UPS) para el servidor y la pantalla de cocina.	comandas activas y paro total del restaurante.	
Disposición al Cambio (Cultura Organizacional): El personal operativo (meseros y cocineros) mostrará una actitud cooperativa durante las pruebas piloto y la fase de levantamiento de requerimientos.	Boicot pasivo del sistema, datos de retroalimentación sesgados y reportes de "fallos" que en realidad son resistencia al uso de la tecnología.	R-07: Resistencia al cambio y baja adopción tecnológica por el personal operativo.

1.1.3 Entregables del proyecto

Para garantizar la transparencia y la correcta gestión de la configuración, los entregables se clasifican en tres categorías obligatorias: Entregables de Software (Producto), Entregables Técnicos (Ingeniería y Arquitectura) y Entregables de Gestión y Calidad.

A. Entregables de Producto

Representa el software ejecutable y los esquemas de datos listos para su despliegue definitivo en el entorno del restaurante.

ID	Entregable / Documento	Componentes Reales Incluidos	Formato de Entrega	Estado
ENT-PROD-01	Base de Datos Relacional (SCD)	Esquema centralizado en MySQL. Incluye tablas optimizadas de usuarios, roles, catálogo de menú, persistencia de comandos e Id_Mesa.	Script .sql estructurado con diccionario de datos.	COMPLETADO (Listo para producción)
ENT-PROD-02	Sistema de Comanda Digital (Código Fuente)	Aplicación web responsiva completa (Sprints 2 al 7). Incluye lógica backend en PHP (PDO), patrón Observer, interfaces de usuario (Cliente, Cocina, Mesero, Caja, Administrador), suites de seguridad, <i>rate limiting</i> y	Código fuente completo en contenedor Dockerfile .	En proceso

		empaquetado final.		
--	--	--------------------	--	--

B. Entregables de Ingeniería y Gestión de Calidad (Documentación)

Documentos formales y planes de ingeniería que validan el diseño, la mitigación de fallos y garantizan la transferencia tecnológica al cliente.

ID	Entregable / Documento	Componentes Reales Incluidos	Formato de Entrega	Estado Actual
ENT-ING-01	Plan de Gestión de Riesgos	Matriz formal de riesgos (§8.3) con análisis de impacto, probabilidad y estrategias de mitigación implementadas.	Documento digital PDF.	En proceso
ENT-ING-02	Descripción de Arquitectura (AD)	Documentación del diseño arquitectónico del SCD bajo las vistas del estándar ISO/IEC/IEEE 42010:2011.	Documento digital PDF.	COMPLETADO
ENT-ING-03	Plan de Pruebas	Estrategia y resultados de las pruebas unitarias, de integración, seguridad y rendimiento con un <i>Bug Rate</i> verificado < 20%.	Documento digital PDF.	En proceso
ENT-ING-04	Manual Técnico del Sistema	Especificación del diseño lógico (IEEE 1016), diagramas <i>Nassi-</i>	Documento digital PDF.	En proceso

		<i>Shneiderman</i> , diagramas de clases y documentación de la arquitectura PDO/Observer.		
ENT-ING-05	Manual de Usuario y Operación	Guía visual e interactiva paso a paso para los roles del restaurante (Administrador, Mesero, Cocina, Caja).	Documento digital PDF.	En proceso

C. Entregables de Gobernanza Ágil (Histórico de Ciclo de Vida)

Artefactos metodológicos que evidencian la gestión, el ritmo de desarrollo y la transparencia del equipo durante las 16 semanas.

ID	Entregable / Documento	Propósito de Control (IEEE 16326)	Formato / Registro	Estado Actual
ENT-AGI-01	Product Backlog	Listado maestro de historias de usuario que gobernó el alcance del sistema.	Documento PDF.	COMPLETADO
ENT-AGI-02	Sprint Planning (S1 - S8)	Planes de alcance quincenales y compromisos de <i>Story Points</i> asignados.	Minutas de planeación por Sprint.	En proceso
ENT-AGI-03	Daily Scrum (Bitácora)	Registro diario de sincronización del equipo y resolución de impedimentos.	Bitácora técnica de seguimiento.	En proceso

ENT-AGI-04	Sprint Review & Retrospective	Evaluaciones de incrementos de software y lecciones aprendidas de los procesos de desarrollo.	Informes ejecutivos post-sprint.	En proceso
-------------------	--	---	----------------------------------	------------

Criterio de Cierre de Proyecto (Validación IEEE 16326)

Para que el proyecto se declare formalmente en estado **Cerrado y Aceptado**, se deben cumplir las siguientes condiciones de esta sección:

1. El sistema de comanda (ENT-PROD-02) debe cambiar su estatus a completado al finalizar todas sus pruebas e implementaciones.
2. El plan de **Plan de Gestión de Riesgos (ENT-ING-01)** debe cambiar su estatus a **completado tras su última actualización**.
3. **El Plan de Pruebas**
4. **El Product Backlog, Sprint Planning (S1 - S8), Daily Scrum (Bitácora) y Sprint Review & Retrospective deben cambiar su estatus a completado tras su última actualización**
5. El **Manual de Usuario y Operación (ENT-ING-05)** debe cambiar su estatus a *Completado* tras una última prueba de legibilidad con el personal del restaurante.
6. Se debe realizar el despliegue de la **Base de Datos (ENT-PROD-01)** y el **Código Fuente (ENT-PROD-02)** en el servidor local del restaurante utilizando las instrucciones descritas en el Manual Técnico.
7. Se firmará el **Acta de Cierre Final** por parte de los integrantes del equipo y los evaluadores/stakeholders, certificando que el alcance se cumplió sin variaciones descontroladas (*creeping scope*).

1.1.4 Cronograma y presupuesto

El proyecto se rige por un cronograma de 7 Sprints (14 semanas), donde el presupuesto se define en términos de **esfuerzo humano (horas-hombre)**, dado que el desarrollo es de fuente abierta y recursos propios.

A continuación, la **Tabla** detalla la relación entre los hitos temporales y la carga de trabajo estimada para el equipo:

Sprint	Hito / Fases	Esfuerzo (HRS)	Entregables principales	Estado
--------	--------------	----------------	-------------------------	--------

Sprint 1	Iniciación y Arquitectura	40	Backlog.	Completado
Sprint 2	Núcleo y Mesas	40	Módulo de Mesas, Validación, Login, Plan de Pruebas.	Completado
Sprint 3	Flujo de Pedidos	36	UI Cocina, Carrito, Lógica de Pedidos (FIFO).	Completado
Sprint 4	Operación Caja	40	UI Caja, Generación de Tickets, Cálculo de Cambio.	Completado
Sprint 5	Administración	40	Dashboard, CRUD Productos/Inventario, Observer.	Completado
Sprint 6	Calidad y Seguridad	40	Pruebas (Unitarias/Seguridad), Rate Limiting.	Completado
Sprint 7	Integración y Despliegue. Cierre y Aceptación	40	Docker, Pipeline CI/CD, Manuales, Pruebas Rendimiento. Manual de Usuario, Acta de Cierre, Liberación Final.	Proceso

Nota de referencia: El diagrama de Gantt, la distribución de tareas y el registro histórico del flujo de trabajo (capturas de tablero Kanban) se encuentran documentados en el archivo adjunto: "**Anexo B: Cronograma de Sprints y Evidencias de Gestión Ágil**". Este documento es parte integral de la documentación del proyecto y debe consultarse para el control de desviaciones temporales.

Presupuesto y estimación de esfuerzo

Dado que el Sistema de Comanda Digital (SCD) es un proyecto de desarrollo basado en *software* libre y hardware preexistente, el presupuesto se define mediante el **Costo de Esfuerzo en Horas-Hombre**. Esta métrica es el indicador estándar para medir la inversión real de tiempo requerida para completar los entregables identificados en el inventario del proyecto.

Parámetros de estimación

Para el cálculo del esfuerzo se han definido los siguientes parámetros operativos:

- Equipo de trabajo: 3 integrantes
- Capacidad individual: 4 horas efectivas de desarrollo

- Capacidad del equipo: 12 horas-hombre por día
- Periodo de ejecución: 14 semanas (70 días hábiles de desarrollo activo).
- Esfuerzo estimado: 7
- $3(\text{personas}) \times 4(\text{horas}) \times 70(\text{días}) = 840$ horas hombre totales

Distribución del esfuerzo

La distribución del presupuesto de tiempo se ha categorizado según las fases del ciclo de vida del software, permitiendo una trazabilidad clara entre el diseño, la construcción y el aseguramiento de la calidad.

Tabla: Presupuesto de Esfuerzo por Categoría (IEEE 16326)

El esfuerzo total del proyecto fue calculado considerando el número de integrantes del equipo, las horas efectivas de trabajo por día y el periodo total de desarrollo activo.

La ecuación utilizada para la estimación fue la siguiente:

$$E = P \times H \times D$$

Donde:

- E = Esfuerzo total estimado en horas-hombre
- P = Número de personas del equipo
- H = Horas efectivas de trabajo por día
- D = Días hábiles de desarrollo

Sustituyendo los valores del proyecto:

$$E = 3 \times 4 \times 70$$

Resultado:

$$E = 840 \text{ horas-hombre}$$

Los porcentajes asignados a cada categoría del proyecto fueron definidos considerando la complejidad, tiempo requerido y nivel de participación de cada fase dentro del ciclo de vida del software, tomando como referencia prácticas de estimación utilizadas en proyectos de desarrollo bajo IEEE 16326.

La fase de **Construcción (40%)** recibió el mayor porcentaje debido a que concentra las actividades de programación, integración de módulos, desarrollo de frontend y backend, siendo la etapa con mayor carga técnica y operativa.

$$840 \times 0.40 = 336$$

La categoría de **Gestión y Documentación (25%)** contempla actividades administrativas y de seguimiento como planeación, backlog, gestión de riesgos, elaboración de manuales y documentación técnica, las cuales son necesarias durante casi todo el proyecto.

$$840 \times 0.25 = 210$$

La fase de **Arquitectura y Diseño (15%)** incluye el modelado de la base de datos, diseño de arquitectura y documentación IEEE 1016. Aunque es una etapa crítica, su duración es menor comparada con la construcción.

$$840 \times 0.15 = 126$$

El área de **Pruebas y QA (15%)** considera pruebas unitarias, integración y validación funcional para asegurar la calidad del software y reducir errores antes de la entrega final.

$$840 \times 0.15 = 126$$

Finalmente, **Despliegue y Cierre (5%)** corresponde a actividades finales como configuración del entorno, despliegue con Docker, validación de aceptación y entrega del producto.

$$840 \times 0.05 = 42$$

Categoría de Esfuerzo	Horas Totales	% de Esfuerzo	Descripción de Actividades
Gestión y Documentación	210	25%	Planeación, <i>Backlog</i> , Manuales, Riesgos, Actas.
Arquitectura y Diseño	126	15%	SDD (IEEE 1016), Diseño BD.
Construcción (Coding)	336	40%	Desarrollo backend, frontend, lógica y módulos.
Pruebas y QA (Calidad)	126	15%	Pruebas unitarias, de seguridad e integración.
Despliegue y Cierre	42	5%	Docker, Aceptación final y entrega.
TOTAL	840	100%	Esfuerzo Total Requerido

1.2 Evolución del plan

De acuerdo con el estándar IEEE 16326, la planificación de un proyecto de software es un proceso dinámico que debe adaptarse a los hallazgos técnicos durante la ejecución. La evolución de nuestro plan se gestionó mediante dos mecanismos de control:

A. Ciclo de Gestión de Cambios (Change Control)

Ante cualquier desviación identificada en el seguimiento semanal (Sprint Review), se aplicó el siguiente protocolo:

- 1. **Identificación:** Detección de la desviación en el cronograma o esfuerzo.
- 2. **Impact Analysis:** Evaluación del impacto sobre el presupuesto total (las 1,280 horas-hombre).
- 3. **Aprobación:** Revisión por parte del equipo para ajustar las tareas del siguiente Sprint.
- 4. **Registro:** Actualización de los artefactos de gestión (Cronograma y Tableros Kanban)

B. Trazabilidad de la Evolución (Versiones del Plan)

El plan de proyecto no fue una "foto fija", sino un documento con control de versiones. La evolución se resume en la siguiente tabla de hitos de actualización:

Versión	Fecha	Motivo de la Actualización	Impacto en Horas/Esfuerzo
V1.0	02-03-2026	Planificación inicial	Ninguno
V1.1	17-03-2026	Ajuste de tiempos de Sprint	Reasignación de horas
V1.2	20-03-2026	Integración de información de sprint retrospective y gestion de riesgos	Incremento de 2 horas de documentación
V1.3	22-03-2026	Integración de metricas KPIs	Incremento de 2 horas de análisis y seguimiento
V1.4	24-04-2026	Integración de actividades WBS	Ajuste organizacional sin impacto

2. Definiciones

En la siguiente tabla se encuentra la definición de conceptos clave del dominio del sistema

Termino	Definición
Backlog	Artefacto central de gestión ágil de alcance
Bug Rate	Métrica de calidad del proyecto
Burn-down Chart	Herramienta de seguimiento de avance
Ciclo DevOps	Fases del ciclo de vida del proyecto
Criterios de aceptación	Validación de producto contra requisitos
Cronograma de Sprints	Línea base del cronograma
Cycle Time	Métrica de eficiencia del equipo
Definition of Done (DoD)	Estándar de calidad para entregables
Entregable	Producto tangible de cada sprint
Historia de usuario	Unidad funcional de alcance
IEEE 16326	Estándar que rige el PMP
KPI	Indicadores de rendimiento del proyecto
Lead Time	Métrica de tiempo total de entrega
Plan de gestión de proyecto (PMP)	El mismo documento
Prioridad	Criterio de ordenamiento del backlog
Product Owner	Rol responsable del valor del producto
Riesgo	Eventos potenciales con plan de mitigación
Rol de usuario	Perfiles de acceso del sistema
Scrum	Metodología de gestión del proyecto
Scrum Máster	Rol que facilita el proceso
Sprint	Unidad de tiempo del cronograma
Sprint Velocity	Métricas de capacidad del equipo
Story point	Unidad de estimación de esfuerzo
Throughput	Métrica de productividad
UAT	Validación con el cliente
Versionado semántico	Gestión de releases y entregas

3. Referencias

1. IEEE, *IEEE Standard for Project Management in IT/Software Projects*, IEEE Std 16326-2009, 2009.
2. ISO/IEEE, *Systems and software engineering — Architecture description*, ISO/IEC/IEEE 42010:2011, 2011.
3. Documento de Diseño de Arquitectura (TDD), Backlog de Producto y Guía de Estilos de Interfaz

4. Contexto del Proyecto

Antecedentes y justificación

El Sistema de Comanda Digital (SCD) surge como respuesta a la necesidad de optimizar la comunicación operativa entre el área de atención al cliente y el área de cocina en establecimientos gastronómicos de mediana escala. Los métodos tradicionales (papel) presentan cuellos de botella en la gestión de pedidos, errores en la captura de órdenes y una falta de trazabilidad en los tiempos de despacho.

Entorno operativo y tecnologico

El proyecto se desarrolla bajo un ecosistema de **software libre**, aprovechando tecnologías de código abierto para asegurar la escalabilidad y reducir costos de licenciamiento. El entorno operativo objetivo es un modelo cliente-servidor con despliegue en contenedores (*Docker*), garantizando la portabilidad entre diferentes infraestructuras de red local.

Factores críticos de éxito

Para que el SCD sea considerado un proyecto exitoso, se han definido los siguientes factores:

- **Integridad de Datos:** Garantizar que no existan inconsistencias entre el pedido realizado por el mesero y el recibido en cocina.
- **Latencia de Comunicación:** Tiempo de respuesta del sistema inferior a 2 segundos bajo carga concurrente.
- **Usabilidad:** Curva de aprendizaje para el personal operativo inferior a 2 horas.
- **Mantenibilidad:** Código estructurado bajo estándares de diseño (Patrones de diseño) que permitan futuras actualizaciones por terceros.

Restricciones del proyecto

- **Recursos:** El desarrollo se realiza con recursos de equipo autogestionados (sin presupuesto de capital para infraestructura comercial).
- **Hardware:** El sistema debe ser capaz de operar en hardware preexistente
- **Alcance Temporal:** Ciclo de vida estricto de 16 semanas, dividido en 8 Sprints de desarrollo incremental.

4.1 Modelo de proceso

Modelo de ciclo de vida: Iterativo-incremental híbrido

El proyecto adopta un enfoque que combina la planificación temporal de Scrum con el flujo continuo de Kanban, integrado dentro de un marco de gestión predictiva conforme a la ISO/IEC/IEEE 16326:2019.

Características:

Iteraciones. - Ciclos de desarrollo de aproximadamente 2 semanas para cada sprint, con entregables incrementales al final de cada iteración.

Flujo de trabajo. - Tablero de Kanban para gestionar el estado de las actividades a realizar, con límites de trabajo en progreso para optimizar el flujo de trabajo. Sirve como mecanismo principal de transferencia de información entre los procesos técnicos y los procesos de gestión.

Reunión quincenal. - Revisión formal de avance del sprint, evaluación de cambios, ajuste de la planificación del PMP y priorización del backlog. Actúa como Sprint review y punto de control de gestión. Así mismo, al terminar el Sprint review se realiza el Sprint Planning.

Reuniones operativas. - Entre reuniones quincenales, el equipo se coordina según las necesidades para resolver problemas que van surgiendo, sin imponer reuniones innecesarias.

Gestión de cambios. - Integrada en la reunión quincenal, con análisis de impacto en alcance, cronograma y presupuesto, siguiendo el proceso de control de cambios del PMP.

Daily Scrum. - Cada día, los desarrolladores rellenan en el Daily Scrum, lo que hicieron ese día e indican si tuvieron o tienen algún problema, el problema se tratará con el Scrum Master y este deberá indicar si se ha resuelto el problema.

Sprint Planning. - Se indica cuáles serán las tareas a realizar en el siguiente sprint.

Hitos y línea base. - Al final de cada sprint, se realiza una retrospectiva donde el equipo analiza el desempeño del sprint, identifica mejoras y ajusta el proceso para el siguiente ciclo. El entregable incremental (PMP, Reportes de Sprint, backlog, etc.) se somete a una revisión que permite establecer una línea base técnica antes de pasar a la siguiente fase de construcción.

Inicio y cierre. - El modelo inicia con una fase de “inspección” para definir las historias de usuario con base en los requisitos iniciales y para definir la organización del equipo de trabajo y termina con una fase de “transición de cierre” para el despliegue final en el restaurante.

4.1.2 Justificación del modelo

Este modelo nos permite mantener la predictibilidad necesaria para la gestión formal del proyecto mientras se beneficia de la flexibilidad del flujo de Kanban para adaptarse a imprevistos sin la rigidez de las reuniones Scrum puras. El tablero Kanban proporciona visibilidad en tiempo real del estado del proyecto, facilitando la toma de decisiones en las reuniones quincenales.

“El desarrollo seguirá los estándares de codificación definidos en la sección 5.4.4 de este plan, y la calidad de los productos de trabajo se asegurará mediante la técnica de definition of Done (DoD), actuando como criterio de aceptación objetivo para cada tarea del tablero Kanban.

4.2 Mejora del proceso

PROBLEMA IDENTIFICADO	CAUSA	ACCIÓN DE MEJORA	RESPONSABLE	SPRINT DE IMPLEMENTACIÓN	ESTADO
CONFIGURACIÓN DE DOCKERS	Falta de memoria en equipos, no configurados	Apoyo del equipo de desarrollo	Oswaldo Ramírez	Sprint 1	Completado
MESAS DUPLICADAS	Falta de control de concurrencia	Desarrollar lógica para evitar selección simultánea	Uriel Vargas	Sprint 3	Pendiente
LIBERACIÓN DE MESAS NO AUTOMÁTICA	Lógica incompleta	Implementar liberación automática al finalizar pedido	Uriel Vargas	Sprint 3	Pendiente
CONTROL DE ROLES INCOMPLETO	Implementación parcial	Finalizar sistema de autenticación y permisos	Oswaldo Ramírez	Sprint 3	Pendiente
ERRORES EN VISUALIZACIÓN DEL MENÚ	Problemas en interfaz	Optimizar interfaz gráfica	Jesús Vicente	Sprint 2	Completado

4.3 Plan de Infraestructura

Cada desarrollador trabajará principalmente desde su dispositivo personal previamente configurado con las herramientas necesarias para el desarrollo del proyecto.

En caso de ser necesario el equipo utilizará cubículos o salones disponibles dentro de la institución para realizar trabajo colaborativo y pruebas integradas.

Infraestructuras tecnológicas

- Trello para la administración visual de tareas mediante Kanban.
- GitHub como repositorio centralizado del código y la documentación
- Docker para estandarizar el entorno de desarrollo entre todos los integrantes.
- OneDrive como sistema complementario de almacenamiento y respaldo de documentos.
- Red local Wi-fi del restaurante para permitir la comunicación entre tablets, caja, cocina y dispositivos administrativos durante las pruebas y operación del sistema.

Herramientas de ingeniería

- Visual Studio Code como entorno principal de desarrollo.
- OpenCode para pruebas y validaciones del sistema.
- Composer y npm para la administración de dependencias.
- MySQL como sistema gestor de base de datos.

Sistemas habilitadores

- GitHub permitirá mantener control de versiones, integración continua y respaldo del código fuente.
- Docker permitirá que todos los desarrolladores trabajen sobre una misma configuración de entorno, reduciendo problemas de compatibilidad.
- La red local del restaurante permitirá la comunicación entre dispositivos para sincronizar pedidos, estados y actualizaciones en tiempo real.

Responsables

Docker. - Jesús Vicente Rios Carrera

- Mantener actualizadas las imágenes Docker.
- Verificar el correcto funcionamiento de los contenedores.
- Asegurar que todos los integrantes tengan acceso a la versión más reciente del entorno.

Tablero trello. - George Edmundo Osornio Tapia, Guadalupe Yosabeth Perez Estrada.

- Actualizar tareas.

- Administrar prioridades.
- Supervisar el flujo de trabajo del proyecto.

OpenCode. - Oswaldo García Ramírez.

- Coordinar pruebas.
- Validar integraciones.
- Verificar el correcto funcionamiento del sistema en el entorno de pruebas.

GitHub. - Todo el equipo

4.4 Métodos, herramientas y técnicas

Metodologías de Desarrollo

Para el desarrollo del proyecto se adoptó una combinación de las metodologías **Scrum** y **DevOps**, debido a que permiten una gestión iterativa del trabajo y una integración continua entre desarrollo, pruebas y despliegue.

La metodología Scrum fue seleccionada porque facilita la organización del proyecto mediante sprints, backlog, reuniones de seguimiento y retrospectivas, permitiendo una administración flexible de cambios y prioridades. Además, favorece la trazabilidad de actividades y la colaboración entre los integrantes del equipo.

Por otro lado, DevOps fue implementado para automatizar procesos de integración, pruebas y despliegue utilizando contenedores Docker y herramientas de automatización. Esta metodología permitió reducir errores de configuración, mejorar la estabilidad del entorno y acelerar el ciclo de retroalimentación.

La combinación de Scrum y DevOps fue elegida debido a que proporciona mayor adaptabilidad y control del proyecto respecto a metodologías tradicionales secuenciales, las cuales presentan menor flexibilidad ante cambios frecuentes en requisitos o funcionalidades.

Herramientas Utilizadas

Sistema Operativo — Fedora 44

Se utilizó **Fedora 44 (Forty Four)** como sistema operativo principal debido a su estabilidad, compatibilidad con herramientas modernas de desarrollo y soporte actualizado para tecnologías de contenedores y automatización.

La versión Fedora 44 fue seleccionada porque incorpora versiones recientes del kernel Linux, compatibilidad estable con Docker Engine 29 y soporte optimizado para Node.js

22 y herramientas DevOps modernas. No se eligieron versiones anteriores debido a limitaciones de compatibilidad y soporte menos actualizado, mientras que versiones posteriores podrían presentar características experimentales o inestabilidad en determinadas herramientas.

XAMPP 8.0.30-0

Se empleó **XAMPP 8.0.30-0** como entorno legacy de desarrollo local para mantener compatibilidad con módulos previamente desarrollados en PHP y MariaDB.

La versión utilizada fue seleccionada por ofrecer estabilidad y compatibilidad con el código heredado del sistema. Versiones anteriores presentaban menor soporte para bibliotecas modernas, mientras que versiones posteriores podían generar incompatibilidades con componentes legacy existentes.

Docker Engine 29.4.2

Se utilizó **Docker Engine 29.4.2** para la contenerización del sistema y la estandarización del entorno de ejecución.

Esta versión fue seleccionada debido a su estabilidad, mejoras de rendimiento y compatibilidad con Docker Compose. Su implementación permitió aislar dependencias, garantizar portabilidad y reducir conflictos entre entornos de desarrollo y producción.

No se utilizaron versiones anteriores debido a limitaciones en soporte de redes y volúmenes, mientras que versiones posteriores aún no contaban con suficiente validación en el entorno del proyecto.

Docker Compose 5.1.2

Se implementó **Docker Compose 5.1.2** para la orquestación de servicios y contenedores del sistema.

La herramienta fue seleccionada debido a que permite administrar múltiples servicios mediante archivos de configuración centralizados, simplificando el despliegue y mantenimiento del sistema.

La versión 5.1.2 fue elegida por su compatibilidad directa con Docker Engine 29.4.2 y por ofrecer una configuración más estable respecto a versiones anteriores.

PHP 8.2

Se utilizó **PHP 8.2** como lenguaje backend principal dentro del entorno Docker.

La versión 8.2 fue seleccionada debido a las mejoras de rendimiento, seguridad y tipado respecto a PHP 8.0 y versiones anteriores. Asimismo, proporciona mayor compatibilidad con bibliotecas modernas y herramientas actuales de pruebas automatizadas.

No se utilizaron versiones más antiguas debido a limitaciones de rendimiento y soporte, mientras que versiones posteriores no habían sido completamente estabilizadas al momento del desarrollo.

MySQL 8.0

Se implementó **MySQL 8.0** como sistema gestor de base de datos dentro del entorno Docker.

Esta versión fue seleccionada por ofrecer mejoras en rendimiento, optimización de consultas y seguridad respecto a versiones anteriores. Además, mantiene compatibilidad con herramientas de contenerización y administración modernas.

Node.js 22.22.2

Se utilizó **Node.js 22.22.2** como entorno de ejecución JavaScript para herramientas auxiliares y automatización.

La versión 22 fue seleccionada debido a su soporte LTS, mejoras de rendimiento y compatibilidad con herramientas modernas como Playwright y servidores MCP.

No se eligieron versiones anteriores debido a limitaciones en dependencias y compatibilidad con herramientas recientes de automatización.

npm 10.9.7

Se empleó **npm 10.9.7** como gestor de paquetes para la instalación y administración de dependencias JavaScript del proyecto.

Esta versión fue seleccionada por mantener compatibilidad directa con Node.js 22 y proporcionar mejoras en resolución de dependencias y rendimiento.

Playwright 1.59.1

Se implementó **Playwright 1.59.1** para automatizar pruebas End-to-End del sistema.

La herramienta fue seleccionada debido a su capacidad para automatizar navegadores modernos, capturar evidencia visual y validar flujos completos de usuario en diferentes módulos del sistema.

La versión utilizada ofrece mejor compatibilidad con Chromium y mayor estabilidad respecto a versiones anteriores.

PHPUnit 10.5

Se utilizó **PHPUnit 10.5** para la ejecución de pruebas unitarias sobre componentes backend desarrollados en PHP.

La versión 10.5 fue seleccionada por ser compatible con PHP 8.2 y proporcionar mejoras en cobertura de pruebas, validación de errores y rendimiento respecto a versiones previas.

TestSprite MCP 0.0.37

Se implementó **TestSprite MCP 0.0.37** como motor de pruebas automatizadas asistidas por inteligencia artificial.

Esta herramienta fue seleccionada debido a su capacidad para generar planes de prueba automáticos, ejecutar validaciones en la nube y generar reportes interactivos con métricas, capturas y videos.

La versión utilizada fue elegida por mantener compatibilidad con cline y Playwright MCP dentro del entorno configurado del proyecto.

Markdownify MCP 1.0.4

Se utilizó **Markdownify MCP 1.0.4** para convertir reportes HTML y PDF generados durante las pruebas automatizadas a formato Markdown.

Esta herramienta permitió mantener documentación homogénea y facilitar la integración de reportes dentro de los artefactos del proyecto.

Git

Se implementó **Git** como sistema de control de versiones para gestionar cambios, ramas y respaldos del código fuente.

Git fue seleccionado debido a su amplia adopción en entornos de desarrollo colaborativo y su capacidad para mantener trazabilidad de cambios, historial de versiones y recuperación de código.

Técnicas Utilizadas

Contenerización

Se aplicó la técnica de contenerización mediante Docker para garantizar portabilidad, aislamiento de dependencias y uniformidad entre entornos de desarrollo y producción.

Automatización de pruebas

Se utilizaron técnicas de automatización de pruebas mediante Playwright, PHPUnit y TestSprite MCP para validar funcionalidades críticas del sistema y reducir errores durante el proceso de desarrollo.

Integración continua

Se implementaron prácticas de integración continua mediante Git, Docker y pruebas automatizadas, permitiendo validar cambios frecuentes y mantener estabilidad en el proyecto.

Control de versiones

Se aplicó control de versiones utilizando Git para registrar cambios del sistema, facilitar la colaboración del equipo y mantener respaldo histórico de modificaciones.

Pruebas End-to-End (E2E)

Se utilizaron pruebas End-to-End mediante Playwright para simular escenarios reales de interacción de usuarios con el sistema, incluyendo autenticación, pedidos, cocina y caja.

Pruebas unitarias

Se aplicaron pruebas unitarias con PHPUnit para validar el funcionamiento individual de módulos backend y detectar errores en etapas tempranas del desarrollo.

4.4.1 Herramientas del Gestion y seguimiento - Trello

Uso del Tablero Kanban

Se utilizará la herramienta **Trello** como plataforma de gestión visual de tareas mediante la metodología Kanban, debido a su facilidad para organizar actividades, monitorear avances y mantener trazabilidad durante el desarrollo del proyecto.

El tablero Kanban permitirá:

- Visualizar el estado actual de las actividades del Sprint.
- Asignar responsables a cada tarea.
- Establecer prioridades de trabajo.
- Controlar el flujo de actividades durante el ciclo de desarrollo.
- Registrar comentarios y seguimiento de incidencias.
- Mantener evidencia del progreso del proyecto.

La selección de Trello se realizó debido a su facilidad de uso, accesibilidad multiplataforma y capacidad de colaboración en tiempo real. Asimismo, permite mantener una administración organizada de tareas sin requerir infraestructura adicional compleja.

Durante cada Sprint se manejará un promedio aproximado de 10 tareas activas, distribuidas según la complejidad funcional y la carga de trabajo del equipo de desarrollo.

Cada integrante trabajará normalmente entre 3 y 4 tareas simultáneas, dependiendo de la prioridad, criticidad y duración estimada de cada actividad.

Cada tarjeta dentro del tablero Kanban deberá contener como mínimo:

- Responsable asignado.
- Nivel de prioridad.
- Estado actual de la tarea.
- Descripción funcional.
- Comentarios de seguimiento.
- Evidencia o referencias relacionadas.
- Fecha estimada de finalización.

Definition of Done (DoD)

Para considerar una tarea como finalizada dentro del Sprint, deberá cumplir obligatoriamente con los siguientes criterios de aceptación:

- Código implementado y funcional.
- Integración correcta dentro del repositorio GitHub.
- Revisión de código entre integrantes del equipo.
- Validación funcional por parte del Scrum Master o Product Owner.
- Pruebas funcionales ejecutadas y aprobadas.
- Validación dentro del entorno Docker o herramientas de ejecución configuradas.

- Actualización del estado de la tarea en Trello.
- Registro de evidencias y observaciones relevantes.

Únicamente después de cumplir todos los criterios establecidos en la Definition of Done (DoD), la actividad podrá marcarse como completada dentro del tablero Kanban.

En caso de que el desarrollador responsable no actualice la tarea, el Scrum Master o Product Owner podrá realizar la actualización correspondiente después de verificar la correcta implementación y validación de la funcionalidad.

4.5 Plan de aceptación del producto

Criterios de Aceptación

El sistema será considerado aceptado cuando cumpla satisfactoriamente con los requisitos funcionales, operativos y de rendimiento definidos para el entorno del restaurante.

Como criterio principal de aceptación, el sistema deberá ser capaz de procesar un pedido desde la tablet asignada a la mesa, registrar la información, enviarla automáticamente a cocina y caja, procesar el cobro correspondiente y actualizar el estado de la orden en un tiempo menor a 2 segundos bajo condiciones normales de operación dentro de la red local del establecimiento.

$t < 2$ segundos

El tiempo de respuesta fue definido considerando la necesidad operativa de mantener un flujo continuo de atención al cliente y evitar retrasos durante la toma y procesamiento de pedidos. Asimismo, el sistema deberá garantizar:

- Correcta sincronización entre módulos.
- Disponibilidad de información en tiempo real.
- Persistencia adecuada de datos.
- Integridad de pedidos y transacciones.
- Estabilidad operativa durante pruebas continuas.

Métodos de Aceptación

La aceptación del sistema se realizará mediante la aplicación de diferentes métodos de validación funcional y operativa, incluyendo:

- Pruebas funcionales de módulos.
- Demostración operativa del sistema.
- Inspección de funcionalidades administrativas.
- Validación de procesos de caja y cocina.
- Revisión de manuales técnicos y de usuario.

- Simulación de escenarios reales de operación.
- Validación de rendimiento dentro de la red local.

Estos métodos permitirán verificar que el sistema cumple con los requisitos establecidos y que su comportamiento es consistente dentro del entorno de trabajo esperado.

Plan de Pruebas de Aceptación

Se realizará una jornada de validación denominada “Prueba en Vivo”, en la cual se simulará el flujo operativo completo del restaurante para comprobar el funcionamiento integral del sistema antes de emitir el acta final de aceptación.

La validación se ejecutará utilizando los siguientes recursos:

- Tablets Android.
- Laptops con Windows.
- Dispositivos móviles de prueba.
- Red local Wi-Fi del entorno operativo.
- Entorno Docker configurado para ejecución del sistema.

Durante esta fase se evaluará:

- Generación de pedidos.
- Modificación de órdenes activas.
- Envío automático de pedidos a cocina.
- Sincronización entre módulos.
- Cobro y registro de pagos en caja.
- Actualización automática de estados.
- Impresión de tickets.
- Liberación y cierre de pedidos.
- Integridad de información almacenada.
- Tiempo de respuesta del sistema.

Acuerdo Formal de Aceptación

Los criterios y métodos de aceptación serán revisados y aprobados conjuntamente entre el equipo de desarrollo y el responsable operativo del restaurante antes de iniciar las pruebas finales del sistema.

Cualquier observación, incidencia o solicitud de ajuste identificada durante la validación deberá registrarse formalmente y resolverse antes de la aprobación definitiva del producto.

Evidencia de Aceptación

La aceptación final del sistema quedará respaldada mediante la siguiente documentación y evidencia técnica:

- Reportes de pruebas funcionales.
- Registro de incidencias detectadas.
- Resultados de validación operativa.
- Capturas y evidencias de funcionamiento.
- Reportes generados por herramientas de prueba.
- Validación de tiempos de respuesta.
- Acta de conformidad y visto bueno del responsable del restaurante.

Toda la evidencia recopilada será almacenada como parte de la documentación oficial del proyecto para asegurar trazabilidad y respaldo del proceso de aceptación.

4.6 Organización del proyecto

Estructura Organizacional

La organización del proyecto se definió utilizando una estructura modular basada en funcionalidades, con el propósito de facilitar el desarrollo, mantenimiento, integración y seguimiento de actividades durante el ciclo de vida del software.

El proyecto se desarrollará bajo un esquema colaborativo utilizando la metodología Scrum, permitiendo que las responsabilidades y actividades puedan ajustarse dinámicamente durante cada Sprint según las necesidades operativas, prioridades del backlog y carga de trabajo del equipo.

Aunque las responsabilidades pueden variar durante el desarrollo, cada integrante trabajará principalmente sobre un módulo funcional específico para mantener trazabilidad, especialización técnica y control de cambios.

La división modular del sistema permite:

- Reducir dependencias entre componentes.
- Facilitar pruebas e integración continua.
- Mejorar la mantenibilidad del sistema.
- Distribuir eficientemente las tareas del Sprint.
- Incrementar la trazabilidad de cambios y errores.

Organización por Módulos Funcionales

Módulo de Autenticación y Control de Acceso

Este módulo corresponde al sistema de autenticación de usuarios y control de acceso al sistema.

Su función principal es validar la identidad de cada usuario mediante credenciales de acceso asociadas a un rol específico dentro del restaurante.

Cada usuario deberá autenticarse utilizando:

- Nombre de usuario o identificador.
- Contraseña asignada.
- Rol correspondiente dentro del sistema.

El módulo controlará el acceso a funcionalidades específicas según el perfil autorizado, permitiendo restringir operaciones de acuerdo con el nivel de permisos definido para cada rol.

Los roles considerados dentro del sistema incluyen:

- Administrador.
- Mesero.
- Cocina.
- Caja.

La implementación de este módulo fue separada del resto del sistema debido a que representa un componente crítico de seguridad y control operativo, permitiendo centralizar la validación de sesiones, permisos y autenticación.

Asimismo, esta separación modular facilita:

- El mantenimiento del sistema.
- La escalabilidad futura.
- La administración de permisos.
- La integración con otros módulos funcionales.
- La validación independiente mediante pruebas unitarias y funcionales.

Asignación de Responsabilidades

Durante cada Sprint, las actividades serán distribuidas entre los integrantes del equipo considerando:

- Complejidad técnica.
- Prioridad funcional.
- Dependencias entre módulos.
- Tiempo estimado de desarrollo.
- Carga de trabajo actual.

La asignación de tareas será gestionada mediante el tablero Kanban del proyecto, permitiendo mantener seguimiento continuo del avance, estado y responsables de cada actividad.

En caso de requerirse soporte o integración entre módulos, los integrantes podrán colaborar temporalmente en componentes distintos a su módulo principal, manteniendo coordinación mediante reuniones de seguimiento y revisión del Sprint.

4.6.1 Interfaces externas

Participación de Actores Externos

El proyecto contempla la participación de actores externos con el propósito de validar requisitos, verificar el funcionamiento operativo del sistema y asegurar que las funcionalidades desarrolladas cumplan con las necesidades reales del entorno del restaurante.

La interacción con estos participantes permitirá obtener retroalimentación funcional y operativa durante las diferentes fases del proyecto, especialmente en actividades de validación y aceptación del sistema.

Dueño del Restaurante

El dueño del restaurante actuará como principal responsable de la validación de requisitos funcionales y operativos del sistema.

Sus responsabilidades dentro del proyecto incluyen:

- Validar necesidades operativas del restaurante.
- Revisar y aprobar requerimientos funcionales.
- Verificar que los módulos desarrollados cumplan con los procesos reales del negocio.
- Participar en revisiones de avance y validación de funcionalidades.
- Aprobar criterios de aceptación del producto.
- Emitir la conformidad final del sistema previo a su liberación.

La participación del responsable del restaurante es fundamental para asegurar que el sistema mantenga alineación con los procesos administrativos y operativos reales del entorno de trabajo.

Usuarios Finales

Los usuarios finales participarán como consultores funcionales durante las pruebas de usabilidad y validación operativa del sistema.

Su intervención permitirá evaluar el comportamiento del sistema en escenarios reales de uso, verificando la facilidad de operación, tiempos de respuesta y comprensión de las interfaces.

Los usuarios finales colaborarán principalmente en:

- Pruebas de usabilidad.
- Simulación de flujos operativos reales.
- Validación de procesos de pedidos.
- Verificación de funciones de cocina y caja.
- Evaluación de interacción con tablets y dispositivos móviles.

- Identificación de errores o mejoras operativas.

La retroalimentación proporcionada por los usuarios finales será utilizada para realizar ajustes funcionales y mejorar la experiencia de uso antes de la aceptación definitiva del producto.

Comunicación y Retroalimentación

La comunicación con actores externos se realizará mediante reuniones de revisión, demostraciones operativas y sesiones de validación funcional durante los Sprints definidos en el proyecto.

Toda observación, incidencia o recomendación identificada durante estas actividades será registrada y documentada para mantener trazabilidad y control de cambios dentro del proceso de desarrollo.

4.6.2 Interfaces internas

Comunicación Interna del Proyecto

Las interfaces internas del proyecto corresponden a los mecanismos de coordinación, comunicación y colaboración establecidos entre los integrantes del equipo de desarrollo para asegurar la correcta ejecución de actividades durante cada Sprint.

La comunicación interna se realizará mediante reuniones Scrum, seguimiento en tablero Kanban y coordinación técnica continua entre los responsables de cada módulo funcional.

Estas interfaces permiten:

- Coordinar actividades de desarrollo.
- Gestionar dependencias entre módulos.
- Resolver bloqueos técnicos y operativos.
- Mantener trazabilidad de tareas.
- Validar avances funcionales.
- Facilitar la integración continua del sistema.

Scrum Master

El Scrum Master será responsable de facilitar la comunicación entre los integrantes del equipo y asegurar el cumplimiento de la metodología Scrum durante el desarrollo del proyecto.

Sus principales responsabilidades incluyen:

- Coordinar reuniones Daily Scrum.
- Identificar y eliminar bloqueos técnicos u operativos.
- Supervisar el avance de actividades del Sprint.
- Mantener actualizado el flujo de trabajo en el tablero Kanban.
- Facilitar la colaboración entre integrantes.
- Verificar el cumplimiento de los procesos definidos.

El Scrum Master actuará como facilitador del equipo, promoviendo la organización y continuidad del desarrollo.

Product Owner

El Product Owner actuará como enlace principal entre el dueño del restaurante y el equipo de desarrollo.

Sus responsabilidades incluyen:

- Gestionar y priorizar el Product Backlog.
- Validar requerimientos funcionales.
- Comunicar necesidades operativas al equipo técnico.
- Revisar funcionalidades desarrolladas.
- Participar en validaciones y revisiones de Sprint.
- Aprobar entregables funcionales.

El Product Owner garantizará que las funcionalidades implementadas mantengan alineación con los objetivos operativos y administrativos del restaurante.

Desarrolladores Frontend y Backend

Los desarrolladores frontend y backend mantendrán coordinación técnica continua para asegurar la correcta integración entre interfaces visuales, lógica de negocio y servicios del sistema.

La comunicación diaria entre desarrolladores permitirá:

- Integrar funcionalidades entre módulos.
- Validar flujos operativos del sistema.
- Sincronizar cambios de interfaz y backend.
- Resolver errores de integración.
- Garantizar compatibilidad entre componentes.
- Mantener consistencia en la experiencia de usuario.

La coordinación técnica será especialmente importante en los módulos relacionados con:

- Tablets de pedidos.
- Sistema de autenticación.
- Cocina.
- Caja y pagos.
- Gestión de órdenes.
- Sincronización en tiempo real.

Mecanismos de Coordinación

La coordinación interna del proyecto se realizará mediante:

- Reuniones Daily Scrum.
- Sprint Planning.
- Sprint Review.
- Sprint Retrospective.

- Tablero Kanban en Trello.
- Control de versiones con Git.
- Comunicación técnica entre responsables de módulos.

Estos mecanismos permitirán mantener seguimiento continuo del estado del proyecto y asegurar la correcta integración de los componentes desarrollados.

4.6.3 Autoridades y Responsabilidades

Para asegurar que el trabajo se realice según lo planeado, se asignan los módulos mencionados con anterioridad a roles específicos mediante una Matriz de Responsabilidades lo que nos permite identificar quien ejecuta la actividad, quien aprueba el trabajo realizado, quién debe ser consultado y quién debe mantenerse informado sobre el avance del módulo.

Actividad / Módulo	Responsable	Rinde cuentas / Aprueba	Consultado	Informado
Autenticación (Login)	Jesús Vicente Rios Carrera	Scrum Master	QA	Dueño del restaurante
Gestión de Mesas	Uriel Vargas Angeles	Scrum Master	Meseros	Dueño del restaurante
Menú Digital	Oswaldo García Ramirez	Product Owner	Dueño del restaurante	QA
Módulo de Meseros	Uriel Vargas Angeles	Scrum Master	Meseros	QA
Módulo de Cocina	Jesús Vicente Rios Carrera	Scrum Master	Cocina / Chef	QA
Módulo de Caja	Oswaldo García Ramirez	Product Owner	Dueño del restaurante	Caja
Panel de Administración	Uriel Vargas Angeles	Product Owner	Dueño del restaurante	QA
Inventario y Stock	Jesús Vicente Rios Carrera	Product Owner	Administración	QA

Integración de módulos	Equipo de desarrolladores	Scrum Master	Product Owner	Dueño del restaurante
Pruebas funcionales	Oswaldo García Ramirez	Scrum Master	QA	Equipo de desarrollo
Validación del flujo completo	Scrum Master y Product Owner	Product Owner	Dueño del restaurante	Equipo de desarrollo
Gestión de Docker y entorno	Jesús Vicente Rios Carrera	Scrum Master	Equipo de desarrollo	Product Owner
Gestión de Trello y seguimiento	Scrum Master y Product Owner	Scrum Master y Product Owner	Equipo de desarrollo	Dueño del restaurante
Gestión de documentación	Scrum Master y Product Owner	Scrum Master y Product Owner	Equipo de desarrollo	Product Owner

Cada responsable deberá garantizar que las funcionalidades asignadas cumplan con los criterios de aceptación, se integren correctamente al sistema y pasen por revisión de código, pruebas funcionales y validaciones antes de considerar finalizadas.

La asignación de responsabilidades podrá ajustarse entre sprints dependiendo de la carga de trabajo y necesidades del proyecto.

5. Planificación del proyecto

5.1 Inicialización del proyecto

Inicio del Proyecto

La inicialización del proyecto corresponde a la fase en la cual se definieron los objetivos generales, alcance funcional, recursos técnicos y organización inicial del sistema de comanda digital.

Durante esta etapa se establecieron las bases operativas y metodológicas necesarias para garantizar un desarrollo estructurado y alineado con las necesidades del restaurante.

La fase de inicialización permitió:

- Definir el alcance preliminar del sistema.

- Identificar requerimientos funcionales y operativos.
- Establecer la metodología de trabajo.
- Seleccionar herramientas y tecnologías.
- Asignar roles y responsabilidades.
- Estimar tiempos y esfuerzo del proyecto.
- Definir el entorno de desarrollo y pruebas.

Definición del Alcance

Durante la inicialización se definió que el sistema permitiría gestionar digitalmente el flujo operativo del restaurante mediante módulos interconectados para:

- Gestión de pedidos.
- Autenticación de usuarios.
- Administración de mesas.
- Cocina.
- Caja y pagos.
- Control de estados de órdenes.
- Sincronización en tiempo real.

El alcance fue delimitado considerando las necesidades operativas del restaurante y los recursos disponibles para el desarrollo.

Identificación de Requerimientos

En esta fase se realizaron reuniones iniciales con el responsable del restaurante para identificar procesos operativos, necesidades funcionales y problemas existentes dentro del flujo tradicional de atención.

Como resultado, se definieron requerimientos relacionados con:

- Rapidez en toma de pedidos.
- Reducción de errores manuales.
- Sincronización entre cocina y caja.
- Control de usuarios mediante roles.
- Visualización de estados de pedidos.
- Automatización de procesos operativos.

Los requerimientos obtenidos fueron registrados y organizados dentro del Product Backlog inicial.

Selección de Metodología

Se seleccionó la metodología Scrum complementada con prácticas DevOps debido a la necesidad de mantener un desarrollo iterativo, flexible y con integración continua.

La metodología Scrum fue elegida para organizar el trabajo mediante Sprints y facilitar el seguimiento de actividades, mientras que DevOps permitió automatizar pruebas, despliegues y validaciones mediante herramientas de contenerización y automatización.

Configuración del Entorno de Desarrollo

Durante la inicialización se preparó el entorno técnico necesario para el desarrollo del sistema, incluyendo:

- Instalación y configuración de Fedora 44.
- Configuración de Docker y Docker Compose.
- Preparación del entorno PHP y Node.js.
- Configuración de base de datos MySQL.
- Inicialización del repositorio Git.
- Configuración de herramientas de pruebas automatizadas.
- Preparación de entorno Docker para validaciones funcionales.

Estas configuraciones permitieron garantizar uniformidad y estabilidad en el proceso de desarrollo.

Organización del Equipo

El equipo de trabajo fue organizado bajo una estructura colaborativa, asignando responsabilidades iniciales para:

- Scrum Master.
- Product Owner.
- Desarrollo Frontend.
- Desarrollo Backend.
- Validación y pruebas.

La asignación inicial de responsabilidades permitió distribuir módulos funcionales y mantener control del avance del proyecto desde las primeras etapas.

Estimación Inicial

Durante la inicialización también se realizó una estimación preliminar del esfuerzo y tiempo requerido para el proyecto.

La estimación fue calculada considerando:

- Número de integrantes.
- Horas efectivas de desarrollo.
- Duración de Sprints.
- Complejidad funcional.
- Actividades de pruebas y documentación.

El esfuerzo total estimado del proyecto fue definido en 840 horas-hombre distribuidas entre las diferentes fases del ciclo de vida del software.

$$E=3\times4\times70=840$$

Validación Inicial

Antes de iniciar el desarrollo formal, los objetivos, requerimientos y alcance del sistema fueron revisados junto con el responsable del restaurante para asegurar viabilidad técnica y alineación con las necesidades operativas identificadas.

La aprobación inicial permitió dar inicio formal al proyecto y comenzar la planificación de los primeros Sprints.

5.1.1 Plan de estimación

Técnica de estimación. - La técnica principal de estimación utilizada será Planning Poker, permitiendo estimar el esfuerzo de las historias de usuario mediante consenso del equipo de desarrollo.

La unidad principal de esfuerzo técnico serán los Story Points, los cuales representan la complejidad, riesgo y tiempo aproximado requerido para completar cada historia de usuario.

Las estimaciones de tiempo del proyecto se establecen considerando distintos niveles de incertidumbre y confianza:

Optimista (4 meses), Más probables (5 meses), Pesimista (6 meses).

Estas estimaciones consideran la disponibilidad parcial del equipo, la carga académica, posibles problemas técnicos y riesgos relacionados con integración y despliegue.

Plan de actualización de estimaciones

Al término de cada sprint, se identifica cuántas historias de usuario se han cubierto, velocidad del equipo (velocity), el esfuerzo invertido, incidencias encontradas y cumplimiento de objetivos del sprint. Y con base a esta información se actualizarán:

- Cronograma.
- Esfuerzo restante.
- Estimaciones futuras.
- Métricas de desempeño como el Schedule Performance Index (SPI).

Las estimaciones podrán modificarse conforme avance el proyecto utilizando datos históricos obtenidos durante los sprints anteriores.

5.1.2 Plan de personal

Perfiles requeridos:

- Scrum Master: 1
- Product Owner: 1
- Desarrolladores: 3

Habilidades requeridas

El equipo de desarrollo deberá contar con conocimientos intermedios en:

- PHP
- JavaScript
- HTML y CSS
- MySQL
- Manejo básico de Docker
- Control de versiones con GitHub
- Arquitectura MVC

Adicionalmente, será necesario contar con habilidades de:

- Trabajo colaborativo
- Resolución de problemas
- Adaptación al cambio
- Documentación técnica

Estrategia de adquisición de personal

La integración del equipo se realizó de forma interna entre los compañeros de clase. Y la asignación de roles se realizó considerando:

- Interés personal
- Experiencia previa
- Afinidad con las responsabilidades del proyecto

Plan de formación inicial

Inicialmente se revisó la documentación previa del sistema de comanda digital y posteriormente se definieron las herramientas y tecnologías que serán utilizadas durante el desarrollo.

La mayoría de las herramientas a utilizar ya eran conocidas por todo el equipo; sin embargo, fue necesaria capacitación adicional en:

- Docker
- OpenCode
- Configuración de entornos de desarrollo

La capacitación se realiza mediante:

- Videos tutoriales
- Documentación oficial
- Pruebas Prácticas
- Apoyo colaborativo entre integrantes

En caso de dificultades técnicas, cualquier integrante del equipo podrá apoyar a otro desarrollador para resolver dudas o problemas relacionadas con el entorno de trabajo.

Estrategía de retención y motivación del equipo

La estrategia de motivación del equipo se basa en el cumplimiento de objetivos académicos, desarrollo de competencias profesionales, aprendizaje de herramientas modernas y fortalecimiento de habilidades prácticas en desarrollo de software colaborativo.

5.1.3 Recursos

Hardware:

- Computadoras personales
- Tablets / Celulares para pruebas
- Laptops con Windows

Software:

- Visual Studio Code
- Docker
- OpenCode
- Trello
- MySQL
- Composer
- npm

Licencias:

Todas las herramientas utilizadas durante el proyecto serán empleadas bajo licencias gratuitas, comunitarias o académicas, evitando costos adicionales y riesgos legales relacionados con software propietario.

Equipos de prueba:

Las pruebas del sistema se realizarán utilizando.

- Tablets Android
- Laptops con Windows
- Computadoras personales
- Dispositivos móviles del equipo de desarrollo

Recursos financieros:

El proyecto será desarrollado mediante autofinanciación de los integrantes del equipo. Los recursos financieros contemplan gastos operativos mínimos relacionados con:

- Consumo eléctrico
- Conexión a internet
- Transporte
- Alimentación durante jornadas de trabajo presencial

Recursos de infraestructura:

Se dispondrá de:

- Una mesa de trabajo por cada dos desarrolladores.
- Acceso a toma de corriente eléctrica, de preferencia con dos o más conectores.
- Conexión Wi-Fi
- Espacios de trabajo dentro de la universidad

Generalmente el trabajo se realizará desde casa; sin embargo, por comodidad o necesidad se utilizarán:

- Cubículos
- Biblioteca
- Salones disponibles dentro de la universidad

Sistemas habilitadores

Conforme a IEEE 15288, se consideran sistemas habilitadores:

- GitHub para control de versiones y respaldo del proyecto,
- Docker para estandarización del entorno,
- y Trello para gestión del flujo de trabajo.

Estos sistemas facilitan:

- integración continua,
- coordinación del equipo,
- despliegue,
- trazabilidad,
- y administración del proyecto.

Gestión de reservas para contingencias

El Scrum Master y el Product Owner deberán mantener respaldos actualizados de:

- Documentación.
- Plantillas.
- Reportes.
- Archivos críticos del proyecto.

Las copias deberán almacenarse en:

- Su dispositivo de trabajo.
- Su nube.
- GitHub.
- De ser posible pero no necesario, drive físico.

Los desarrolladores deberán mantener control de versiones mediante GitHub y conservar respaldos locales de su trabajo.

Estas actividades se relacionan con la gestión de configuración y respaldo del proyecto.

5.1.4 Entrenamiento Se realizará capacitación interna en:

- Manejo de la base de datos
- Docker

- OpenCode
- GitHub
- Trello

El entrenamiento tendrá una duración aproximada de una semana, dedicando un día intensivo a cada herramienta para:

- Resolver dudas.
- Configurar entornos.
- Validar funcionamiento.

Validación del entrenamiento

La efectividad de la capacitación se validará mediante:

- Creación exitosa de contenedores Docker.
- Conexión correcta al entorno de desarrollo.
- Realización del primer push al repositorio GitHub.
- Ejecución satisfactoria de pruebas básicas del sistema.

En caso de que algún integrante presente dificultades técnicas, otro desarrollador brindará apoyo para asegurar que todos los miembros puedan utilizar correctamente las herramientas del proyecto.

5.2 Planes de trabajo

5.2.1 Alcance

Implementación de gestión de mesas. - La gestión de mesas permite la asignación y control de mesas disponibles dentro del restaurante a cada una de las tabletas que se usarán para la generación de pedidos.

El sistema evitará asignaciones duplicadas y permitirá liberar mesas al finalizar el servicio.

Implementación del menú digital. - Permite la visualización dinámica de productos, categorías, precios y disponibilidad de platillos para la generación de pedidos desde las tabletas.

Implementación de módulo de meseros. - Permite a los meseros visualizar pedidos activos (ya pagados), modificar prioridades de producción y liberar pedidos finalizados.

Implementación del módulo de cocina. - Permite visualizar pedidos en orden de llegada o prioridad, así como actualizar el estado de preparación y liberar pedidos completados.

Implementación del módulo de caja. - Permite realizar el cobro de pedidos, generar tickets, consultar cuentas activas y registrar pagos realizados por los clientes.

Implementación del panel de administración. - Permite la gestión de usuarios, control de inventario, gestión de productos, configuración del sistema y generación de reportes administrativos.

5.2.2 Actividades (WBS)

Revisión de documentación anterior del sistema de comanda digital

Producto de trabajo: Comprender y comparar el proyecto desde el punto de vista del equipo de trabajo anterior contra las necesidades reales del cliente.

Recursos: Documentación anterior.

Duración: Tres días.

Responsable: Scrum Master, Product Owner, Desarrolladores

Criterio de aceptación: Todos han revisado la documentación.

Revisión de estándares

Producto de trabajo: Comprensión de la planeación y organización del proyecto.

Recursos: Estándares en pdf, cuaderno estándares, profesor.

Duración: Tres días, uno para cada estándar.

Responsable: Scrum Master, Product Owner, Desarrolladores

Criterio de aceptación: Todos entienden y saben cuales son los estándares con los que se trabajara.

Product Backlog

Producto de trabajo: Borrador de Backlog

Recursos: Documentación anterior

Duración: Dos días

Responsable: Scrum Master y Product Owner

Criterio de aceptación: Se genera el primer borrador del backlog para comenzar a asignar tareas al equipo de desarrollo.

Configuración de Dockers

Producto de trabajo:

Recursos: Computadoras, corriente eléctrica, Docker, YouTube

Duración: 7 días.

Responsable: Equipo de desarrollo

Criterio de aceptación: Todos los equipos están configurados.

Documentación, Daily Scrum, Sprint Review, Sprint Planning, Sprint Retrospective

Producto de trabajo: Plantillas para cada documento relacionado al sprint.

Recursos: Plantillas de sprint.

Duración: 2 días.

Responsable: Scrum Master, Product Owner

Criterio de aceptación: Las plantillas cuentan con los campos necesarios para crear los reportes.

Documentación, Plan de pruebas

Producto de trabajo: Borrador de plan de pruebas y plantilla para datos y resultados de prueba.

Recursos: Plantillas de pruebas, test automatizados.

Duración: 3 días.

Responsable: Guadalupe Yosabeth Perez Estrada

Criterio de aceptación: El borrador contiene las pruebas y las plantillas necesarias para documentar las pruebas a realizar.

Paquete de Pruebas

Producto de trabajo: Conjunto de pruebas funcionales, pruebas de integración automatizadas.

Recursos: Plan de pruebas, OpenCode, Docker, GitHub, dispositivos de pruebas.

Duración: 4 días por sprint.

Responsable: Equipo de desarrolladores

Criterio de aceptación: Las funcionalidades críticas del sistema se ejecutan correctamente sin errores durante las pruebas.

Paquete de trabajo (Administración)

Producto de trabajo: CRUD productos

Recursos: Visual Studio Code, GitHub, Docker, MySQL

Duración: 3 días

Criterio de aceptación: El administrador puede crear, editar, eliminar y visualizar productos correctamente.

Responsable: Uriel Vargas Angeles

Producto de trabajo: Modificar precios

Recursos: Visual Studio Code, GitHub, Docker, MySQL

Duración: 2 días.

Criterio de aceptación: Los precios se actualizan correctamente y los cambios son reflejados en el menú.

Responsable: Oswaldo Ramirez García

Producto de trabajo: Categorías

Recursos: Visual Studio Code, Docker, MySQL

Duración: 2 días.

Criterio de aceptación: El administrador puede crear, editar y visualizar categorías de productos correctamente.

Responsable: Jesús Vicente Rios Carrera

Producto de trabajo: Usuarios y roles

Recursos: PHP, MySQL, Visual Studio Code, Docker.

Duración: 3 días.

Criterio de aceptación: Cada usuario puede acceder únicamente a las funcionalidades correspondientes a su rol.

Responsable: Jesús Vicente Rios Carrera

Producto de trabajo: Asignar mesas

Recursos: Visual Studio Code, Docker, Tablets android

Duración: 3 días.

Criterio de aceptación: Las mesas pueden asignarse correctamente a dispositivos sin duplicidad.

Responsable: Uriel Vargas Angeles

Producto de trabajo: Recetas por porción

Recursos: MySQL, Visual Studio Code, Docker.

Duración: 1 semana.

Criterio de aceptación: Cada producto queda correctamente relacionado con sus ingredientes y cantidades.

Responsable: Jesús Vicente Rios Carrera

Producto de trabajo: Descontar stock

Recursos: PHP, MySQL, Docker.

Duración: 1 semana.

Criterio de aceptación: El inventario se actualiza automáticamente después de registrar un pedido pagado.

Responsable: Jesús Vicente Rios Carrera.

Producto de trabajo: Modificación manual de stock

Recursos: Visual Studio Code, MySQL, Docker.

Duración: 2 días.

Criterio de aceptación: El administrador puede aumentar o disminuir stock manualmente sin afectar la integridad de los datos.

Responsable: Uriel Vargas Angeles

Producto de trabajo: Notificar bajo stock

Recursos: Observer Pattern, PHP, MySQL.

Duración: 2

Criterio de aceptación: El sistema genera alertas automáticamente cuando el stock baja del mínimo establecido.

Responsable: Jesús Vicente Rios Carrera

Producto de trabajo: Agregar/eliminar mesas

Recursos: Panel administrador, MySQL.

Duración: 4 días

Criterio de aceptación: El administrador puede agregar o eliminar mesas correctamente.

Responsable: Jesús Vicente Rios Carrera

Producto de trabajo: Ver mesas ocupadas

Recursos: JavaScript, PHP, MySQL.

Duración: 3 días.

Criterio de aceptación: El sistema muestra correctamente las mesas ocupadas y disponibles.

Responsable: Oswaldo García Ramirez

Paquete de trabajo (Cocina)

Producto de trabajo: Ver pedidos cocina

Recursos: Pantalla de cocina, Docker, Visual Studio Code.

Duración: 4

Criterio de aceptación: La cocina puede visualizar los pedidos pendientes correctamente ordenados.

Responsable: Jesús Vicente Rios Carrera

Producto de trabajo: Estado preparando

Recursos: PHP, JavaScript, MySQL.

Duración: 2 días.

Criterio de aceptación: El estado del pedido cambia correctamente a “En preparación”.

Responsable: Jesús Vicente Rios Carrera.

Producto de trabajo: Estado listo

Recursos: PHP, JavaScript, MySQL.

Duración: 3 días

Criterio de aceptación: La cocina puede marcar pedidos como listos correctamente.

Responsable: Uriel Vargas Angeles

Producto de trabajo: Orden de llegada

Recursos: MySQL, JavaScript, Docker.

Duración: 6 días.

Criterio de aceptación: Los pedidos se muestran en orden cronológico según el momento de pago.

Responsable: Jesús Vicente Rios Carrera

Paquete de trabajo (Caja)

Producto de trabajo: Ver pedidos

Recursos: Visual Studio Code, Docker, MySQL.

Duración: 4 días.

Criterio de aceptación: Caja puede visualizar todos los pedidos pendientes de pago.

Responsable: Oswaldo García Ramirez

Producto de trabajo: Buscar pedidos

Recursos: PHP, MySQL, Visual Studio Code.

Duración: 3 días.

Criterio de aceptación: Caja puede localizar pedidos por mesa o identificador.

Responsable: Jesús Vicente Rios Carrera

Producto de trabajo: Imprimir ticket

Recursos: JavaScript, PHP, impresora virtual/PDF.

Duración: 5 días.

Criterio de aceptación: El ticket muestra correctamente mesa, productos, cantidades, fecha y total.

Responsable: Oswaldo García Ramirez

Producto de trabajo: Calcular cambio

Recursos: JavaScript, PHP.

Duración: 4 días.

Criterio de aceptación: El sistema calcula correctamente el cambio con base en el pago recibido.

Responsable: Uriel Vargas Angeles

Producto de trabajo: Marcar pedido como pagado

Recursos: PHP, MySQL, Docker.

Duración: 3 días.

Criterio de aceptación: El pedido cambia correctamente a estado pagado y se libera para cocina.

Responsable: Jesús Vicente Rios Carrera

Producto de trabajo: Enviar a cocina

Recursos: Fetch API, PHP, MySQL.

Duración: 5 días.

Criterio de aceptación: Los pedidos pagados aparecen automáticamente en cocina y meseros.

Responsable: Oswaldo García Ramirez

Producto de trabajo: Cancelar pedido

Recursos: PHP, MySQL, Docker.

Duración: 3 días.

Responsable: Uriel Vargas Angeles

Criterio de aceptación: Caja puede cancelar pedidos pendientes correctamente.

Producto de trabajo: Registrar cancelados

Recursos: MySQL, PHP.

Duración: 3 días.

Criterio de aceptación: Los pedidos cancelados quedan registrados correctamente en la base de datos.

Responsable: Oswaldo García Rios

Paquete de trabajo (Gestión de Mesas)

Producto de trabajo: Seleccionar mesa

Recursos: Tablets Android, PHP, JavaScript.

Duración: 3 días.

Criterio de aceptación: La mesa seleccionada se asigna correctamente al dispositivo.

Responsable: Uriel Vargas Angeles

Producto de trabajo: Evitar mesas duplicadas

Recursos: MySQL, PHP.

Duración: 4 días.

Criterio de aceptación: El sistema evita asignar una misma mesa a múltiples dispositivos simultáneamente.

Responsable: Uriel Vargas Angeles

Producto de trabajo: Liberar mesa

Recursos: PHP, MySQL.

Duración: 3 días.

Criterio de aceptación: Las mesas pueden marcarse nuevamente como disponibles.

Responsable: Uriel Vargas Angeles

Paquete de trabajo (Pedido)

Producto de trabajo: Visualizar menú

Recursos: PHP, MySQL, Docker.

Duración: 2 semanas

Criterio de aceptación: El cliente puede visualizar categorías, productos y precios correctamente desde la tablet.

Responsable: Jesús Vicente Rios Carrera

Producto de trabajo: Agregar productos al carrito

Recursos: PHP, MySQL, Docker.

Duración: 2 semanas

Criterio de aceptación: Los productos seleccionados aparecen correctamente en el carrito.

Responsable: Oswaldo García Ramirez

Producto de trabajo: Modificar ingredientes

Recursos: PHP, MySQL, Docker.

Duración: 2 semanas

Criterio de aceptación: El cliente puede modificar ingredientes antes de confirmar el pedido.

Responsable: Uriel Vargas Angeles

Producto de trabajo: Eliminar productos al carrito

Recursos: PHP, MySQL, Docker.

Duración: 2 semanas

Criterio de aceptación: Los productos pueden eliminarse correctamente del carrito.

Responsable: Oswaldo Garcia Ramirez

Producto de trabajo: Confirmar pedido

Recursos: PHP, MySQL, Docker.

Duración: 2 semanas

Criterio de aceptación: El pedido se registra correctamente y se envía a caja.

Responsable: Uriel Vargas Angeles

Producto de trabajo: Guardar pedido pendiente

Recursos: PHP, MySQL, Docker.

Duración: 2 semanas

Criterio de aceptación: El sistema guarda correctamente pedidos pendientes de pago.

Responsable: Oswaldo García Ramirez

Producto de trabajo: Volver a ordenar

Recursos: PHP, MySQL, JavaScript.

Duración: 3 días.

Criterio de aceptación: El cliente puede volver a solicitar productos previamente ordenados.

Responsable: Jesús Vicente Rios Carrera

Paquete de trabajo (Mesero)

Producto de trabajo: Ver pedidos

Recursos: Pantalla de meseros, Docker, JavaScript.

Duración: 1 semana.

Criterio de aceptación: Los meseros pueden visualizar correctamente los pedidos activos.

Responsable: Uriel Vargas Angeles

Producto de trabajo: Marcar pedido entregado

Recursos: PHP, JavaScript, MySQL.

Duración: 3 días.

Criterio de aceptación: Los pedidos entregados desaparecen correctamente de la lista activa.

Responsable: Uriel Vargas Angeles

Paquete de trabajo (Calidad)

Producto de trabajo: Integrar módulos

Recursos: GitHub, Docker, Visual Studio Code.

Duración: 2 días por sprint..

Criterio de aceptación: Todos los módulos funcionan correctamente integrados sin errores críticos.

Responsable: Equipo de desarrolladores.

Producto de trabajo: Pruebas funcionales

Recursos: Plan de pruebas, OpenCode, dispositivos.

Duración: 1 semana.

Criterio de aceptación: Las funcionalidades cumplen con los requerimientos establecidos.

Responsable: Oswaldo García Ramirez

Producto de trabajo: Validar flujo

Recursos: Simulación de restaurante, tablets, laptops.

Duración: 2 días.

Criterio de aceptación: El flujo completo desde pedido hasta entrega funciona correctamente.

Responsable: Scrum Master, Product Owner

Paquete de trabajo (Seguridad)

Producto de trabajo: Login por rol

Recursos: PHP, MySQL, Docker.

Duración: 5 días.

Criterio de aceptación: Cada usuario puede iniciar sesión únicamente con sus credenciales válidas.

Responsable: Jesús Vicente Rios Carrera

Producto de trabajo: Restricción por rol

Recursos: PHP Sessions, MySQL.

Duración: 3 días.

Criterio de aceptación: Cada rol accede únicamente a los módulos autorizados.

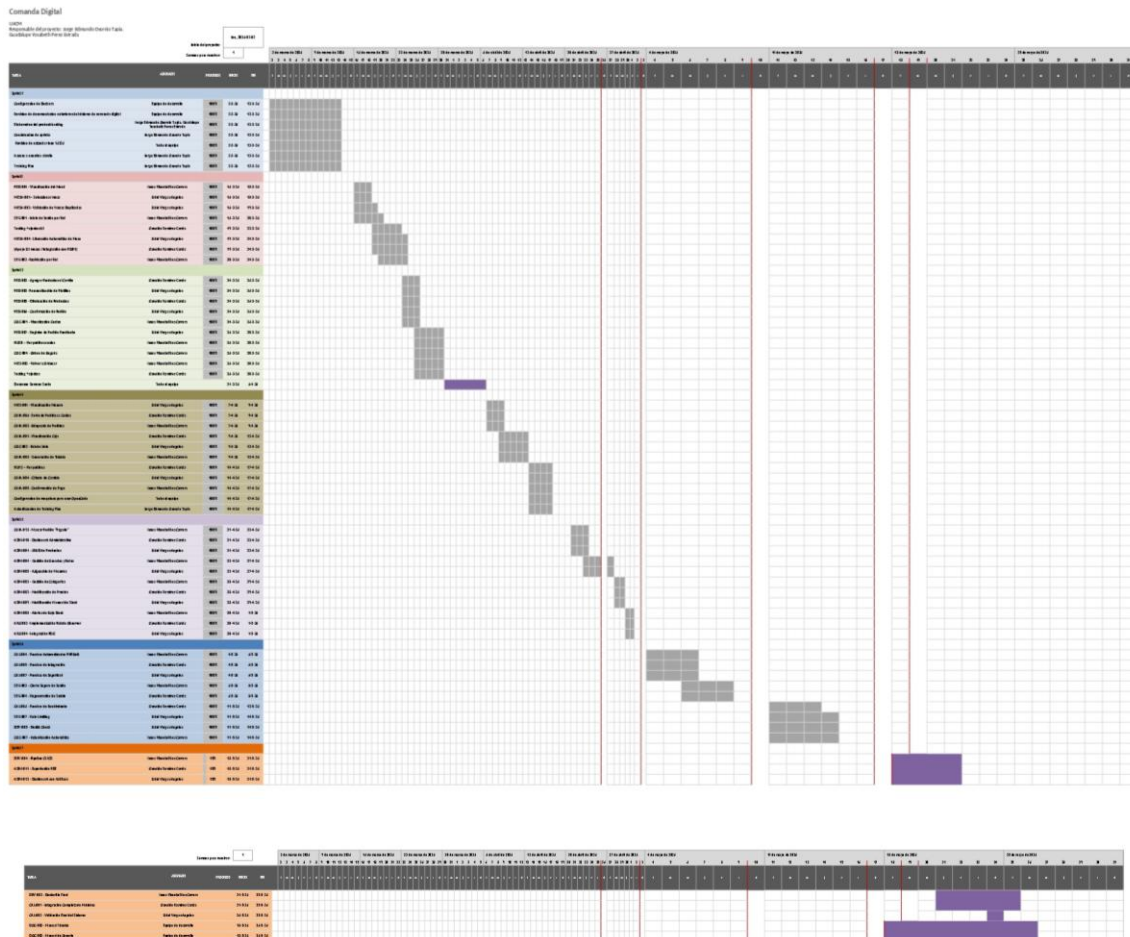
Responsable: Jesús Vicente Rios Carrera

5.2.3 Cronograma y presupuesto

Diagrama de Gantt

El cronograma del proyecto se organiza mediante sprints iterativos de 11 días hábiles, distribuidos entre las actividades de análisis, desarrollo, pruebas, integración y

documentación. El seguimiento del avance se realiza mediante el tablero Kanban en Trello y revisiones periódicas al finalizar cada sprint.



Hitos del proyecto

Hito 1. - Revisión de documentación, configuración del entorno y aprobación del Product Backlog (Sprint 1).

Incluye la revisión de la documentación heredada, definición del backlog inicial, configuración de Docker, acceso a herramientas colaborativas y organización inicial del proyecto.

Hito 2. - Implementación funcional de autenticación, gestión de mesas y generación de pedidos (Sprint 2).

Se completa el sistema de login por roles, asignación de mesas y flujo inicial de generación de pedidos desde las tablets.

Hito 3. - Integración de cocina, meseros y flujo de pedidos pagados (Sprint 3).

Los pedidos ya pagados son enviados correctamente a cocina y meseros, permitiendo visualizar órdenes por prioridad y orden de llegada.

Hito 4. - Implementación del módulo de caja e integración con inventarios (Sprint 4 y 5).

Se habilita el cobro de pedidos, impresión de tickets, cálculo de cambio, cancelación de pedidos y actualización de inventario mediante Observer Pattern.

Hito 5. - Validación integral, pruebas finales y documentación del sistema (Sprint 6 y 7).

Incluye pruebas funcionales, integración completa de módulos, corrección de errores, validación del flujo completo del restaurante y generación de documentación final.

Presupuesto

El presupuesto del proyecto se estima de forma académica, considerando que el sistema fue desarrollado mediante autofinanciación de los integrantes del equipo y utilizando herramientas gratuitas o de licencia comunitaria.

Categorías del presupuesto

Personal

El costo principal corresponde a horas-hombre estimadas para:

- Scrum Master.
- Product Owner.
- Equipo de desarrollo.
- Actividades de pruebas y validación.

El esfuerzo técnico se estima mediante Story Points y Planning Poker.

Infraestructura

Incluye:

- Consumo eléctrico.
- Servicio de internet.
- Uso de computadoras personales.
- Tablets Android utilizadas para pruebas.
- Uso de red Wi-Fi local durante simulaciones.

Software y herramientas

Se utilizaron herramientas gratuitas:

- Docker Community Edition.
- GitHub.
- Trello.
- Visual Studio Code.
- OpenCode.
- MySQL Community.

Por lo tanto, no existen costos de licenciamiento comercial.

Reserva de contingencia

Se considera una reserva destinada a:

- Problemas de configuración de Docker.
- Fallas de integración entre dispositivos.
- Errores en inicialización de base de datos.
- Problemas de autenticación y login.

Control presupuestal

El presupuesto será revisado al finalizar cada sprint, comparando:

- Horas estimadas vs horas reales
- Recursos utilizados y línea base aprobada
- Riesgos ocurridos y consumo de reservas

6. Evaluación y control

6.1 Gestión de requisitos

Obtención

En la etapa anterior del proyecto se establecieron los requisitos funcionales y no funcionales, estos se obtuvieron a través de entrevistas con el administrador y meseros y la observación directa del flujo de trabajo en la cocina.

La versión aprobada de dichos requisitos corresponde a la Especificación de Requisitos de Software (SRS).

Historias de usuario

Durante esta etapa, los requisitos funcionales se transformaron en Historias de Usuario organizadas dentro del Product Backlog. Cada historia contiene:

- Prioridad
- Estado
- Valor de negocio
- Story Points
- Responsable

Trazabilidad

Cada Historia de Usuario mantiene relación directa con uno o varios requisitos funcionales definidos en la SRS, permitiendo conservar la trazabilidad del sistema.

Gestión de cambios

Cualquier modificación a un requisito deberá analizar:

- Impacto en alcance
- Impacto en cronograma
- Impacto en costo
- Impacto técnico
- Riesgos asociados

Los cambios serán discutidos durante las reuniones quincenales con stakeholders.

6.2 Control de alcance

Proceso de control de cambios:

Las solicitudes de cambios deberán ser solicitadas formalmente mediante una reunión y posteriormente:

- Ser documentadas formalmente.
- Analizarse técnicamente.
- Evaluar impacto en costo, tiempo y riesgos.
- Ser presentadas durante las reuniones de revisión del sprint.

Nota: Deben evitarse los cambios innecesarios para evitar costos y riesgos

Comité de control de cambios

Las decisiones relacionadas con cambios de alcance serán tomadas por:

- Product Owner

- Scrum Master

Cuando el cambio afecte directamente la operación del restaurante, también se consultará al administrador o dueño.

Para evitar el crecimiento descontrolado del proyecto, el Product Backlog será priorizado continuamente. Solo podrán desarrollarse tareas aprobadas dentro de la WBS.

Las tareas fuera del alcance definido deberán esperar aprobación formal.

6.3 Control de cronograma

Gestión del valor ganado

El cronograma se actualiza cada semana, con base en la monitorización que se lleva a cabo mediante:

- Velocity del equipo
- Porcentaje de historias completado

Actualización del cronograma

El cronograma se revisará semanalmente considerando:

- Tareas terminadas.
- Dependencias pendientes.
- Riesgos técnicos.
- Integración de módulos.

Y con base a los datos recopilados, si se identifica algún retraso se llevarán a cabo acciones correctivas como:

- Redistribución de tareas entre desarrolladores.
- Aplicación de fast-tracking cuando sea posible.
- Priorización de funcionalidades críticas.

Informes de estado

Estos son todos los reportes generados, al inicio, durante y al finalizar cada sprint; los reportes incluirán.

- Avance físico (de ser posible).
- Hitos alcanzados.
- Riesgos detectados.
- Desviaciones del cronograma.
- Actividades pendientes.

6.4 Control de presupuesto

El control presupuestal busca mantener el proyecto dentro de los recursos estimados inicialmente.

Métricas de costo

En cuestiones de costo, se calcularán las horas hombre con una comparación de horas estimadas vs reales.

Aprobación de gastos

Se plantea que el gasto monetario sea de \$0 pesos, pero en caso de ser necesario algún gasto, los gastos deben cubrirse por partes iguales por el equipo. Además, los gastos menores relacionados con pruebas o infraestructura podrán ser aprobados por el Scrum Master y los gastos relacionados con hardware o recursos externos requerirán aprobación del Product Owner.

Gestión de reservas

En cuestión de reservas, no hay reservas.

6.5 Aseguramiento de la calidad

Tiene como objetivo verificar que el proyecto y el producto desarrollado cumplan con los requisitos de calidad definidos en los estándares IEEE 12207, IEEE 15288 e IEEE 16326, así como con las necesidades establecidas por el cliente.

Para cumplir con ello se establece que se tendrán auditorías internas periódicas enfocadas en verificar el cumplimiento de los estándares de codificación establecidos por el equipo, revisar el cumplimiento de las reuniones Scrum y el uso correcto del tablero Kanban. También validar la correcta documentación de cambios, pruebas y entregables.

Así mismo, se confirmará que las funcionalidades implementadas cumplan con las historias de usuario aprobadas.

Las revisiones de calidad incluirán:

Revisión de código por partes antes de integrar cambios a la rama principal.

- Pruebas funcionales.
- Pruebas de integración.
- Pruebas de caja negra.

Estándares aplicados:

- IEEE 12207
- IEEE 15288
- IEEE 16326

Así mismo se aplicará el ciclo de mejora continua PDCA (Planificar auditorías, realizar evaluaciones, actuar sobre los hallazgos y mejorar el proceso).

Tal como se indica, se planificarán auditorías y revisiones, se efectuarán evaluaciones y pruebas; con base a los datos recolectados se identificarán defectos e implementarán acciones correctivas y mejoras en el siguiente sprint.

Como métricas de calidad se utilizarán:

- Densidad de defectos.
- Número de incidencias críticas por sprint.
- Velocity del equipo.
- Resultados de pruebas funcionales y de integración.

6.6 Subcontratistas

El proyecto no contempla la contratación de subcontratistas externos para el desarrollo, pruebas, infraestructura o soporte del sistema.

Todas las actividades relacionadas con el análisis, diseño, desarrollo, pruebas, documentación e implementación serán realizadas directamente por el equipo del proyecto.

Debido a que no existen terceros involucrados en actividades críticas del sistema, no se requiere un plan formal de selección, evaluación o control de subcontratistas.

6.7 Cierre del proyecto

6.7.1 Criterios de cierre

Para que el proyecto pueda darse por terminado, deberán cumplirse los siguientes criterios:

- Cumplimiento del 100% de las Historias de Usuario aprobadas en la línea base de requisitos.

- Todas las tareas del tablero Kanban deben encontrarse en estado “Done” conforme a la Definition of Done (DoD).
- Finalización de pruebas funcionales, pruebas de integración, pruebas de caja negra y pruebas de validación.
- Resolución de todos los errores críticos identificados durante las pruebas.
- Ejecución satisfactoria de una prueba del sistema en el entorno real del restaurante.
- Validación y aceptación formal del sistema por parte del dueño o administrador del restaurante.
- Entrega de documentación técnica y documentación de usuario actualizada.
- Respaldo final del código fuente y documentación en GitHub y almacenamiento en nube.

6.7.2 Actividades de cierre

Acta de cierre. - Documento formal donde el cliente confirma que el sistema cumple con los criterios de aceptación establecidos y acepta la entrega final del producto.

Informe final. - Este documento consolida los resultados obtenidos durante el proyecto, incluyendo el resumen de funcionalidades implementadas, resultados de las pruebas realizadas, comparación entre el presupuesto estimado y real, horas hombre, riesgos ocurridos y lecciones aprendidas.

Análisis de desempeño vs plan. - Se realizará una evaluación comparando el cronograma planteado contra el cronograma real, el presupuesto línea base contra el presupuesto real y el cumplimiento de hitos y objetivos del proyecto.

7. Entrega del proyecto

7.1 Transferencia técnica y operativa del sistema al restaurante

El objetivo es poner el sistema de la comanda digital en funcionamiento dentro del restaurante, asegurando su compatibilidad con la infraestructura disponible y garantizando que el personal pueda utilizarlo correctamente.

La implementación incluirá:

- Instalación del sistema en los equipos y tablets Android utilizados en el restaurante.
- Configuración de la red local Wi-Fi para la comunicación entre módulos.
- Configuración de usuarios y roles iniciales.
- Verificación de conexión entre caja, cocina, meseros y menú digital.
- Validación del funcionamiento del sistema en el entorno real de operación.

La transferencia se considerará completada una vez que el sistema opere correctamente durante el periodo de validación establecido y el cliente apruebe su funcionamiento.

7.2 Estrategia de entrega

La entrega del sistema se realizará de manera gradual para reducir riesgos operativos durante la transición del proceso tradicional al sistema digital.

Durante la primera semana de operación:

- El sistema de comandas digital coexistirá con las comandas en papel.
- El personal podrá utilizar ambos métodos mientras se adapta al nuevo flujo de trabajo.
- El equipo de desarrollo dará seguimiento a errores, dudas y ajustes menores detectados durante el uso real.

La estrategia de despliegue incluye:

1. Instalación y configuración del sistema.
2. Validación interna por parte del equipo.
3. Capacitación al personal.
4. Prueba piloto en entorno real.
5. Liberación final del sistema.

7.3 Entregables finales

Al finalizar el proyecto se entregarán los siguientes elementos:

- Sistema de comanda digital completamente funcional.
- Base de datos configurada.
- Sistema instalado y configurado en tablets Android y equipos del restaurante.
- Manual técnico del sistema.
- Manual de usuario para meseros y administración.
- Documentación del proyecto (PMP, backlog, pruebas y evidencias).
- Resultados de pruebas funcionales y de integración.
- Acta de aceptación y cierre del proyecto.
- Configuración de Docker y entorno de despliegue.
- Código fuente.

7.4 Documentación y capacitación

Se desarrollará documentación técnica y operativa para facilitar el uso y mantenimiento del sistema (documentación anterior).

La documentación incluirá:

- Manual de usuario para meseros.
- Manual de operación para caja.
- Manual administrativo para el gerente o administrador.
- Manual técnico para mantenimiento y despliegue.
- Guía de recuperación básica ante fallos comunes.

La capacitación será presencial y estará enfocada en:

- Uso del menú digital.
- Gestión de pedidos.
- Cobro y generación de tickets.
- Gestión de cocina y pedidos entregados.
- Solución de problemas básicos.

Las sesiones de capacitación incluirán prácticas reales utilizando tablets Android y simulaciones del flujo operativo del restaurante.

7.5 Soporte post-entrega

Después de la entrega del sistema se establecerá un periodo de soporte técnico para atender errores críticos y dudas operativas durante.

El soporte incluirá:

- Corrección de errores críticos.
- Ajustes menores de configuración.
- Asistencia técnica básica.
- Monitoreo inicial del sistema durante la puesta en marcha.

Se establece un periodo de garantía de 4 meses posteriores a la entrega oficial del sistema.

8. Procesos de soporte

8.1 Supervisión y entorno de trabajo

Supervisión del progreso

El Scrum Master realizará seguimiento continuo al avance del proyecto mediante los Daily Scrum, revisiones semanales del tablero Kanban, seguimiento de tareas críticas y el control de hitos y entregables.

El avance será evaluado utilizando métricas como:

- Velocity del equipo.
- Número de tareas completadas.
- Cumplimiento de Story Points por sprint.

Ambiente de trabajo

El equipo trabaja de manera presencial y remota. Las reglas incluyen:

- Commit obligatorio al final de cada jornada.
- Reunión Daily de máximo 15 minutos a las 9:00 AM.
- Revisión de código cruzada (Oswaldo revisa a Uriel, etc.).
- Acceso a GitHub, Docker y Trello.

Seguridad y salud ocupacional

Se establecerán medidas para evitar sobrecarga de trabajo y agotamiento del equipo:

- Distribución equilibrada de tareas.
- Horarios razonables de desarrollo.
- Pausas durante jornadas extensas.
- Comunicación abierta sobre bloqueos o problemas personales que afecten el trabajo.

Bienestar del equipo

Se fomenta la colaboración mutua entre integrantes, por lo que los desarrolladores pueden comunicarse abiertamente (pero de forma respetuosa) entre ellos y con el Scrum Master y Product Owner.

También se fomenta el apoyo tanto técnico como el aprendizaje colaborativo y la resolución temprana de conflictos.

Mecanismos de escalamiento de problemas de ambiente de trabajo

Procedimiento de escalamiento

Problemas técnicos (en el código). - Si un impedimento de esta clase no se resuelve en 48 horas tras ser detectado en el Daily Scrum, el Scrum Master escalará el problema al Product Owner para decidir si se ajusta el cronograma o se adquiere soporte externo.

Problemas técnicos (equipo). - En caso de tener problemas con algún dispositivo de trabajo o equipo, se buscará solucionarlo en un tiempo de 48 horas tras ser reportado al Scrum Master, en caso de superar el tiempo destinado a solución, se buscará un dispositivo diferente previamente configurado para trabajar.

Esto incluye:

- Falta de herramientas.
- Problemas de conectividad.
- Fallas de hardware.
- Problemas de coordinación.

Problemas de tiempo. - Desde el inicio del proyecto se estimó un tiempo aproximado para terminar las tareas y se le asignó un tiempo extra en caso de tener problemas, para evitar cambios o ajustes bruscos en el cronograma.

8.2 Gestión de decisiones

Categorías de decisión. -

Estrategia. -

Registro. -

SPRINT	RIESGO	MITIGACIÓN
SPRINT 1	- Problemas de instalación de Docker - Problemas de instalación de XAMPP	- Instalación en equipo para evitar errores - Pruebas de funcionamiento y verificación de puertos
SPRINT 2	- Mesas duplicadas - Fallos de sincronización - Problema de roles	- Validación - Pruebas - Implementación de control

SPRINT 3	▪ Omisión de inserción de productos al carrito ▪ Fallo de sincronización ▪ Inconsistencias en el stock de ingredientes al restar cantidades de los platillos. ▪ Retrasos o fallos en la visualización en la visualización de pedidos en cocina.	▪ Validación ▪ Pruebas
-----------------	--	---

8.3 Gestión de riesgos

Proceso continuo para identificar y tratar incertidumbres que puedan afectar el éxito del sistema de comanda.

Estrategía de identificación. -

Análisis y priorización. -

Plan de respuesta. -

Seguimiento. -

<i>Riesgo</i>	<i>Impacto</i>	<i>Probabilidad</i>	<i>Plan de Mitigación</i>
<i>Concurrencia de Mesas</i>	Alto	Media	Implementar un "Flag" o estado de bloqueo en la BD cuando una mesa está siendo editada.
<i>Pérdida de Sesión</i>	Medio	Baja	Uso de cookies persistentes y manejo de estados en el lado del servidor.

<i>Inconsistencia de Stock</i>	Alto	Media	Transacciones SQL atómicas para asegurar que el inventario solo se descuenta si el pedido es exitoso.
<i>Caida del servidor local</i>	Alto	Media	Implementación de backups automáticos cada 4 horas en una unidad externa.
<i>Incompatibilidad de tablets</i>	Medio	Medio	Desarrollo basado en diseño responsivo

8.4 Configuración (Git, versiones)

Control de versiones (Git)
Cambios en el sistema

8.5 Gestión de la información

como guardan documentos
Seguridad de datos

Comunicación y publicidad

Mecanismo de transferencia de información

Durante cada reunión quincenal se presenta evidencia objetiva del progreso para que el cliente pueda validar los requisitos. Se documentan las decisiones tomadas sobre el backlog o cambios en el alcance las cuales deben quedar registradas en el historial de cambios del PMP.

Matriz de comunicación del proyecto

Información a comunicar	Frecuencia	Formato/Herramienta	Emisor (Responsable)	Receptor (Stakeholders)
Estado del flujo de trabajo	Tiempo real	Tablero Kanban	Equipo de desarrollo	Scrum Master
Bloqueos o impedimentos	Diario	Daily Scrum / Slack	Desarrolladores	Scrum Master

Demo de incremento funcional	Quincenal	Reunión Presencial	Scrum Master	Product Owner / Dueño
Actualización del PMP y riesgos	Quincenal	Documento compartido	Scrum Master / Product Owner	Dueño
Test	Quincenal	Reunión presencial	Equipo de desarrollo	Scrum Master / Product Owner
Sprint Planning	Quincenal	Reunión presencial	Scrum Master / Product Owner	Equipo de desarrollo
Sprint Review	Diario	Tablero Kanban	Scrum Master / Product Owner	Scrum Master / Product Owner
Sprint Retrospective	Quincenal	Reunión presencial	Scrum Master / Product Owner	Scrum Master / Product Owner / Equipo de desarrollo

8.6 Aseguramiento de calidad (QA)

Revisiones
Estándares

8.7 Medición

Se definieron métricas clave de desempeño (KPIs) con el objetivo de evaluar la eficiencia del equipo de desarrollo durante los sprints. El Throughput permitió medir la cantidad de historias completadas por iteración, obteniendo un promedio de 2 historias por sprint. El Cycle Time reflejó el tiempo promedio de desarrollo por historia, estimado en 2.5 días. Finalmente, el Lead Time midió el tiempo total desde la solicitud hasta la entrega de una funcionalidad, con un promedio de 10 días. Estas métricas permiten monitorear el desempeño del equipo y apoyar la mejora continua del proceso.

Métrica de Velocidad del Sprint (Sprint Velocity)

<i>Sprint</i>	<i>Story Point (SP) Comprometidos</i>	<i>Story Points (SP) Terminados</i>	<i>Eficiencia (Done/Planned)</i>	<i>Observaciones</i>
<i>Sprint 1</i>	7	7	1	Incentivar el trabajo en equipo
<i>Sprint 2</i>	32	32	1	Modificación de SP por observaciones del Product Owner
<i>Sprint 3</i>	18	13	0.72	En revisión
<i>Sprint 4</i>	24			
<i>Sprint 5</i>	18			
<i>Sprint 6</i>	11			
<i>Sprint 7</i>	29			
<i>Sprint 8</i>	47			

Burn-down Chart: trabajo pendiente vs. el tiempo. Permite detectar si el equipo terminará a tiempo antes del cierre del Sprint.

<i>Sprint</i>	<i>Trabajo inicial (Total Backlog SP)</i>	<i>Trabajo Restante (Actual)</i>	<i>Trabajo restante (Ideal)</i>
<i>Sprint 1</i>	200	193	193
<i>Sprint 2</i>	193	161	161
<i>Sprint 3</i>	161	148	142
<i>Sprint 4</i>	-	-	-
<i>Sprint 5</i>	-	-	-
<i>Sprint 6</i>	-	-	-
<i>Sprint 7</i>	-	-	-
<i>Sprint 8</i>	-	-	-

Calidad (Bug Rate): Número de errores reportados en las pruebas de aceptación sobre el total de HU entregadas.

<i>Sprint</i>	<i>Historia de Usuario (HU) Entregadas</i>	<i>Bugs Detectados (QA/UAT)</i>	<i>Tasa de Error (Bug HU)</i>
<i>Sprint 1</i>	7	2	0.28
<i>Sprint 2</i>	8	3	0.37
<i>Sprint 3</i>	6	3	0.5
<i>Sprint 4</i>	-	-	-
<i>Sprint 5</i>	-	-	-
<i>Sprint 6</i>	-	-	-
<i>Sprint 7</i>	-	-	-
<i>Sprint 8</i>	-	-	-

Throughput

SPRINT	HISTORIAS COMPLETADAS
SPRINT 1	7
SPRINT 2	6
SPRINT 3	3

Throughput promedio:

$$P = (7+6+3)/3 = 5.3$$

Resultado: 5.3 historias por sprint

Cycle Time

HISTORIA	DÍAS
HU001	5
HU002	2
HU003	2
HU004	1
HU006	1
HU007	1
HU01	2
HU02	4
HU04	3
HU06	2
HU39	6
HU40	4
HU07	4
HU08	4
HU09	4
HU10	4
HU11	4

Promedio

$$P = (5+2+2+1+1+1+2+4+3+2+6+4+4+4+4+4+4)/17 = 3.17$$

Resultado

$$R = 3.17 \text{ días por historia}$$

Lead Time

Se creo el 30/2/2026

Historia	Días
HU001	6
HU002	3
HU003	3
HU004	2
HU005	3
HU006	2
HU007	2
HU01	2
HU02	7
HU04	11
HU06	2
HU39	6

HU40	14
HU07	18
HU08	18
HU09	18
HU10	En proceso
HU11	En proceso

Promedio:

$$(6+3+3+2+3+2+2+2+7+11+2+6+14+18+18+18)/16 = 7.3$$

Resultado:

7.3 días por historia aproximadamente

8.8 Revisiones y auditorías

Revisiones internas

Revisiones con cliente

8.9 Verificación y validación

- **Pruebas Unitarias (Unit Testing):** Pruebas a funciones específicas de PHP (ej. validar que un descuento del 10% se aplique correctamente al total).
- **Pruebas de Integración:** Verificar que el módulo de "Pedidos" inserte correctamente los datos que el módulo de "Cocina" debe leer.
- **Pruebas de Estrés:** Simular 20 pedidos simultáneos desde diferentes tablets para asegurar que el servidor SQL no colapse.
- **Criterios de Aceptación:** Cada Historia de Usuario debe cumplir con una lista de verificación antes de marcarse como "Hecha" (ej. "El sistema no permite avanzar si no se ha seleccionado al menos un producto").

9. Planes adicionales

10. Anexos

Anexo A – Product Backlog

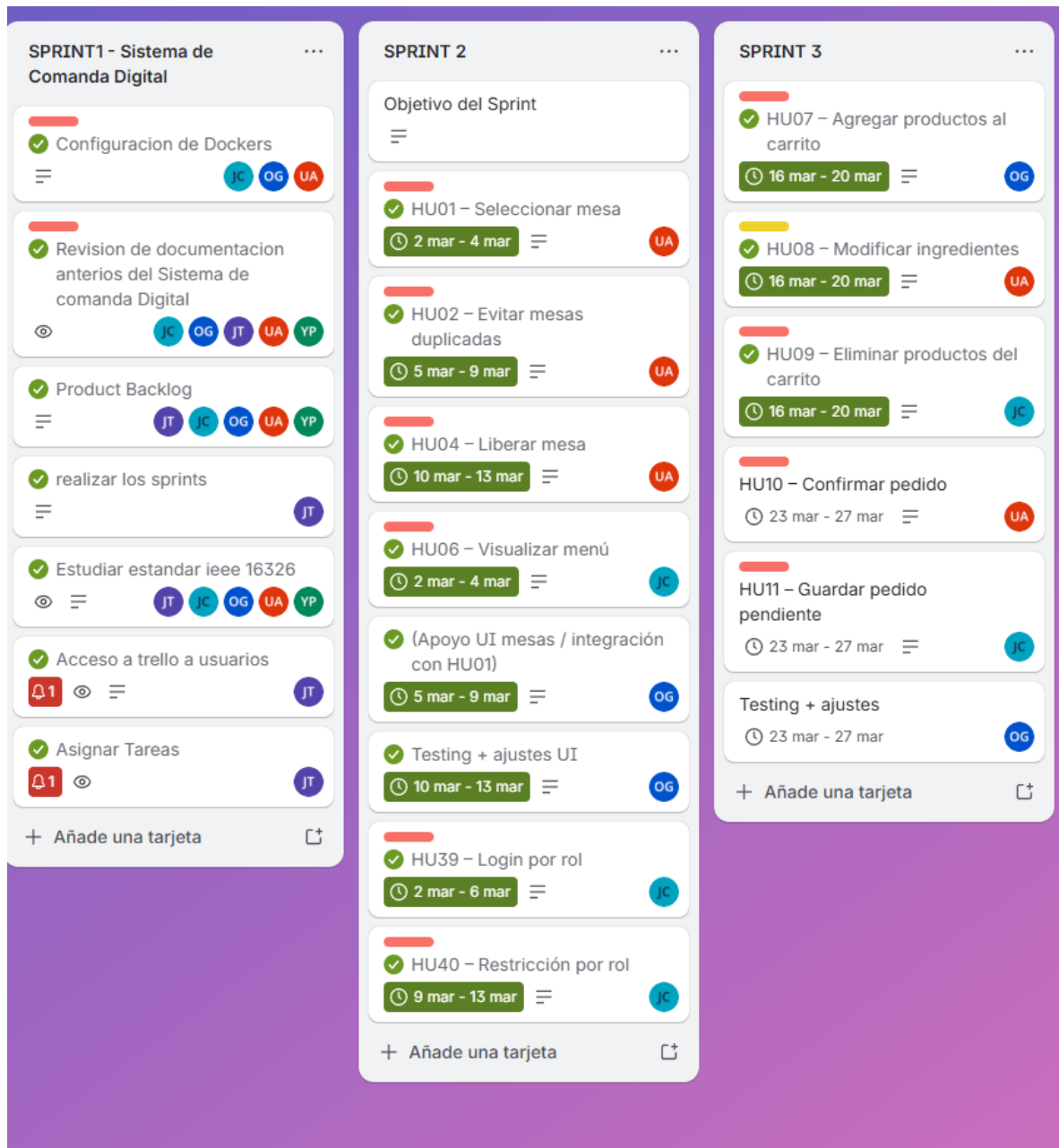
Anexo B – Sprint Backlog

Anexo C – Sprint Review

Anexo E – Sprint Retrospective

Anexo F -KPIs

Anexo G - Kanban



Anexo H – Plan de pruebas

Anexo I – Pruebas